A Guide to Data Entry and Documentation in EpiData

using Manager and EntryClient

by

Myo Minn Oo

Version 2.2.1

Table of Contents

Chapter 1. Introduction to EpiData 6
1.1 What is EpiData? 6
1.2 Features and Usage of the new EpiData 7
1.3 Installing EpiData Manager and EntryClient 7
1.4 Terminology 8
1.5 Help and Documentation 8
1.6 References 9

Chapter 2 Getting Started with EpiData Manager 10
2.1 Opening EpiData Manager 10
2.2 Creating a New Project 11
2.3 Navigating newly created project 11
2.4 Saving and Closing the current Project 14
2.5 Opening existing projects 15
2.6 Example project: Form 1 in Tuberculosis Programme 16

Chapter 3. Creating a codebook 18
3.1 Characteristics of a codebook 18
Name 19
Label 19
Type 19
Range 20
Value labels 20
Notes 20
3.2 A Codebook for Form 1 21
Additional Notes 25

Chapter 4. Designing a dataform 26
4.1 Design tools 27
4.2 Adding headings 28
4.3 Adding variables 30
String variable 30
DMY variable 32
Integer variable 34

Memo variable *36*
4.4 Alignment 37
4.5 Creating derived fields 38
Patient identifier *38*
Combining into a derived variable *40*
4.6 Unique index 42
4.7 Jumping to next variables 43
Jump Value *43*
Designated point (Go To Field) *43*
Reset Value *43*
Demonstration *43*
Chapter 5. Getting started with EntryClient 45
5.1 Opening EpiData EntryClient 45
5.2 Opening a project 45
5.2 Entering values in fields 46
5.3 Navigating records 47
5.4 Printing records 47
5.5 Deleting records 47
Chapter 6. Double data entry and data management 49
6.1 Double data entry and validation 49
Preparation *49*
Double data entry *49*
Validation *52*
Finalization *56*
6.2 Exporting data 57
Stata *58*
CSV File *58*
SPSS *58*
DDI *59*
EPX *59*
6.3 Appending records 60
6.4 File backup 63
Storage drives *63*

Zipped EPX File *64*
6.5 Archiving files with encryption 65
Chapter 7. Data documentation 67
7.1 Report structure 67
7.2 Comparing files for duplicates 67
7.3 Count records 67
7.4 Data content validation 67
Chapter 8. Creating relational database 67
8.1 Relational database 67
8.2 Creating relational dataform 67
8.3 Enter relational data 67
8.4 Deleting and Exporting relational data 67
Chapter 9. User access control system 67
9.1 Setting single password 67
9.2 User access control 67
9.3 Defining roles and rights 67
9.4 Access log Overview 67
9.5 Removing the control 67
Chapter 10. Advanced properties of dataforms 67
Chapter 11. Advanced settings 67
11.1 Version control 67
Chapter 12. EpiData and R 67
Annexure: Shortcut keys 68

[This page is intentionally left blank.]

Chapter 1. Introduction to EpiData

1.1 What is EpiData?

EpiData is a collection of freeware that are specifically designed and developed for quality data entry, data documentation, data management, and basic statistical analysis. Its main applications are in public health surveillance, outbreak investigations and scientific research. The development and distribution of the EpiData software is maintained by the EpiData Association, which is based in Denmark.

The EpiData freeware succeeded its principles from the Epi Info software package was developed by the United States Center for Disease Control and Prevention (US CDC) during the 1980s. In 2000, the US CDC released a new Epi Info version 2000 that used Microsoft Access@ database for data storage. Hence, in order to developing an independent and text-based system, **Jens M Lauritsen** took the initiative of the EpiData project which later grew into a fully developed data entry and documentation software known as the **EpiData entry**. It has several advantages in addition to a standalone freeware which include double entry verification, list of ID numbers in several files, codebook overview of data, date added to backup and encryption procedures.

This freeware, **EpiData Entry**, uses three-file-type system, the so-called QES, REC and CHK triplet. In order to create a data entry project, users have to manually type texts in EpiData's text editor (QES) and later convert into a record file (REC) where data are actually stored. If checks for data validation are desired, record file has to be called in to create checks (CHK). However, most people are not good at manual typing and multiple-file-based system can lead to error if files are not in the same directory. Hence, these two have been major drawbacks of the freeware. After version 3.1 was released, the EpiData Association stopped its further development.

Since 2008, the EpiData Association started developing another similar freeware known as **EpiData Manager and EntryClient** which will be our focus in this book. As the names suggest, **Manger** allows users to create projects and develop dataforms whereas **EntryClient** is solely for data entry and record management. The new system added several features that old **EpiData Entry** lacked, such as single-file system, better user interface, click and drop function to create data entry fields, improved relational data system and extended user access control system. Yet it still maintains the principle of simplicity. Last but not least advantage is its cross-platform compatibility meaning that this freeware can be used on Windows, Mac OS as well as Linux operating systems.

The intention of this book is to give readers a rather practical approach to the EpiData Manager and EntryClient for efficient data entry, documentation and data management. In order to facilitate the learning process, the use of technical terms is minimized. Instructions are also illustrated using a real-world project. It is my hope that this will enable readers to get started using EpiData freeware with minimal problems.

One important thing to note is that this freeware collection was developed and maintained by volunteers on a very limited funding. Without their dedication, EpiData would not be accessible for many people. As a result, documentation and how-to guides are somewhat limited. This is where I hope this book can fill the gap.

1.2 Features and Usage of the new EpiData

The first EpiData software was released in 1999. It has been around for more than 20 years now that many aspects have been changed. The new EpiData provides several advantages over the old entry version. Meta-data and records are stored in a single file with extension "`.epx`", which abandons the previous triplet system. The file is basically a text file written in a special web-programming language called "`eXtensible Markup Language`" (`XML`) which is used to store data using simple text. It has become more graphically oriented. It also supports Unicode (`UTF-8`) system hence non-Latin texts can be displayed. Moreover, a lot of efforts were also put to implement good clinical practice (GCP) principle required for many medical data projects. This means data encryption, detailed logging of events and user access control of data.

The EpiData Manager is a tool for the project manager. Its role is to define data structures, add meta-data, document and export data. Files created are also independent of operating system. Once created you can open the file on any computers that install the freeware. The EntryClient serves only data entry. The data entry personnel are not allowed to change rules or structure while doing data entry.

1.3 Installing EpiData Manager and EntryClient

To download them, go to the EpiData Association's official website, http://www.epidata.dk. Under the download page, a list of options for Manager, EntryClient and Analysis. Manager and EntryClient are available in both 32-bit and 64-bit computer architecture under two operating systems: Mac OS and Linux. For Windows users, an all-in-one installer including EpiData Analysis is available to download and install.

The version at the time of writing this book is 4.6.0 (as of 1st September 2019). There can be drastic changes and discrepancies between the book and future version.

1.4 Terminology

Field refers to variables with certain characteristics such as numeric, decimals, text or date. While data entry, values will be put into these fields corresponding to their pre-specified data types.

Record refers to the combination of fields or variables in a subject or participant.

Dataset refers to a compilation of such records. In EpiData, it also refer to a dataform which holds a number or such records.

Figure 1.4 illustrates the visual representation of these concepts.

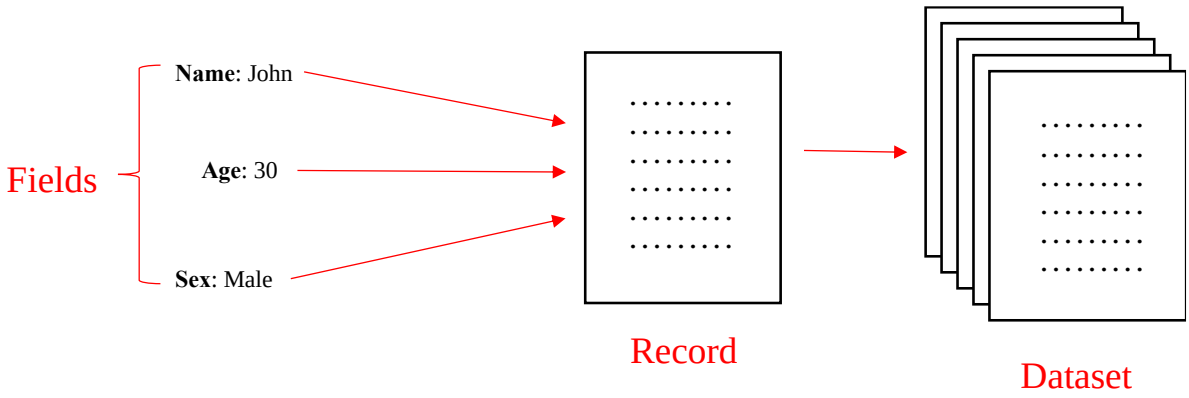


Figure 1.4 Visual representation of field, record and dataset

1.5 Help and Documentation

There are many ways to seek help. First, there are introduction manuals and examples from the website (http://epidata.dk/download/). Second, you can get help from an online forum called "EpiData-list -- EpiData development and support" which lists a number of online subscribers: http://lists.umanitoba.ca/mailman/listinfo/epidata-list. After your subscription to the list, you can access the forum. However, most of the forum administrators and experts who respond to queries also work on a volunteer basis, so their responses to your queries may not be instantaneous. Third, there is also a web-archive for the queries and responses which you can access here: http://lists.umanitoba.ca/pipermail/epidata-list/. Finally, I would also recommend reading the following manuals.

1. Short Introduction to EpiData Manager Version 2.01 J. Lauritsen & T. Christiansen http://www.epidata.dk/downloads/epidatamanagerintro.pdf

2. EpiData EntryClient Short Introduction, Documentation and help file. Version 2.0 J. Lauritsen & T. Christiansen http://www.epidata.dk/downloads/epidataentryclientintro.pdf
3. EpiData Software for Operations Research in Tuberculosis Control: A course developed by the EpiData Association. Hans L. Reider and J. Lauritsen https://tbrieder.org/epidata/epidata.html

1.6 References

1. EpiData Software Freeware: EpiData Flyer General. http://www.epidata.dk/downloads/epidataflyer_general.pdf
2. EpiData Course background by Hans L. Reider: https://tbrieder.org/epidata/course_0-2_background.pdf
3. Short Introduction to EpiData Manager v2.01 J. Lauritsen & T. Christiansen: link http://www.epidata.dk/downloads/epidatamanagerintro.pdf
4. EpiData EntryClient Short Introduction, Documentation and help file. v2.0 J.Lauritsen/T.Christiansen: link http://www.epidata.dk/downloads/epidataentryclientintro.pdf
5. JM. Lauritsen, TB. Christiansen, HL. Rieder, J. Hockin EpiData Analysis Introduction. Http://www.EpiData.dk (2018) http://epidata.dk/downloads/EpiDataAnalysis_Introduction.pdf

Chapter 2 Getting Started with EpiData Manager

2.1 Opening EpiData Manager

As soon as you open the Manager, check the version you just installed. (see Figure 2.1.1). There are three versions: 1) Current version, 2) Public (stable) version, and 3) Test (beta) version. The Current and Test version (as of July 2019) is 4.6.0.0. Public version is still 4.4.2.1.

Old versions or Mac OS version will prompt you with a window checking version online when you open the software. In that case, there is an option to turn that off.

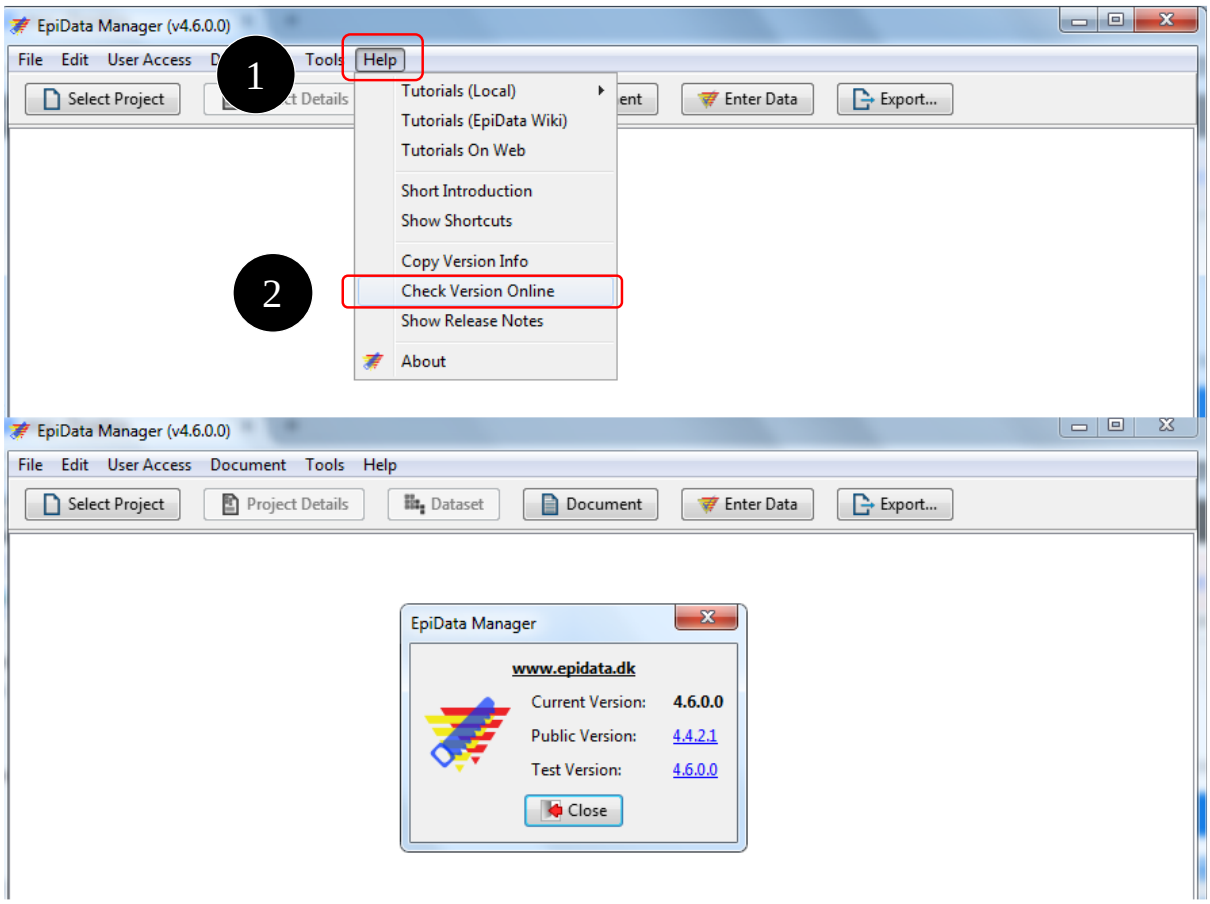


Figure 2.1.1 Checking the EpiData version online

As I mentioned earlier, its advantage is the graphical user interface which is simple and intuitive. Figure 2.1.2 shows menu bar and toolbar. The toolbar is also called **work process toolbar** which provides a generic workflow from project creation and data documentation to data entry and export.

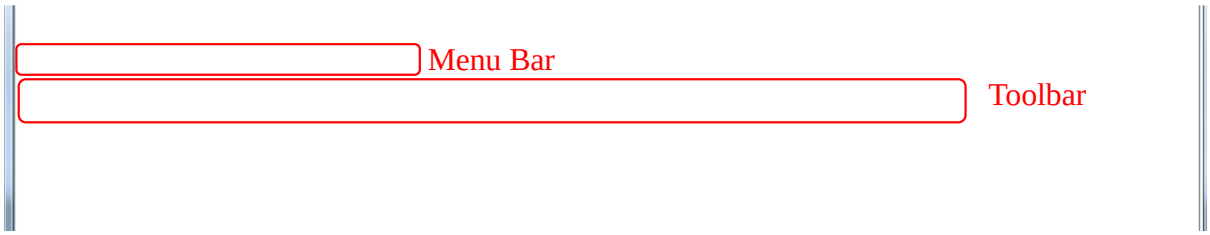


Figure 2.1.2 Interface of EpiData Manager

2.2 Creating a New Project

To create a new blank project, click "**Select Project**" on toolbar or "**File**" on menu bar, and then choose "**New Project**". Alternatively, you can use the keyboard shortcut "`Ctrl + N`" on Windows or "⌘ + `N`" for Mac OS. (See the annexure for a list of shortcut keys)

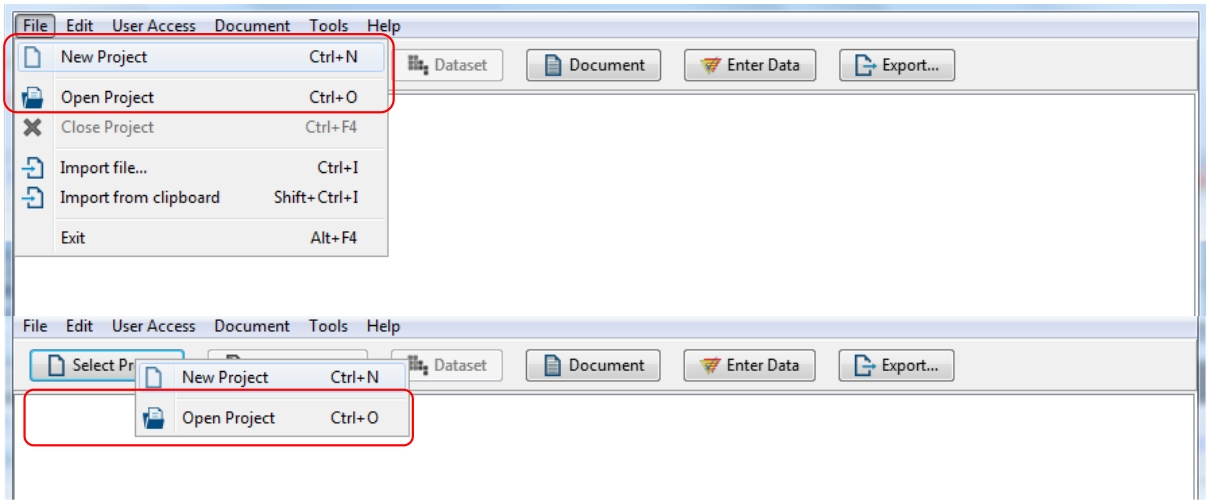


Figure 2.2.1 Creating a new project from menu bar versus from toolbar

2.3 Navigating newly created project

When a new project is created, the screen below the toolbar is split into two parts: a small screen on the left and the big one on the right. Let's call the small one "**Project Tree**" (because later you will see that many sub-dataforms can be grown under the project like branches from a root) and the bigger one "**Study Information**". Below them is a "**status bar**".

On **Study Information** page, there is a welcome tab with a brief instruction on how to create single and relational dataforms. At this moment, we will focus on single dataforms. Relational dataform is an advanced topic which we will learn at a later chapter.

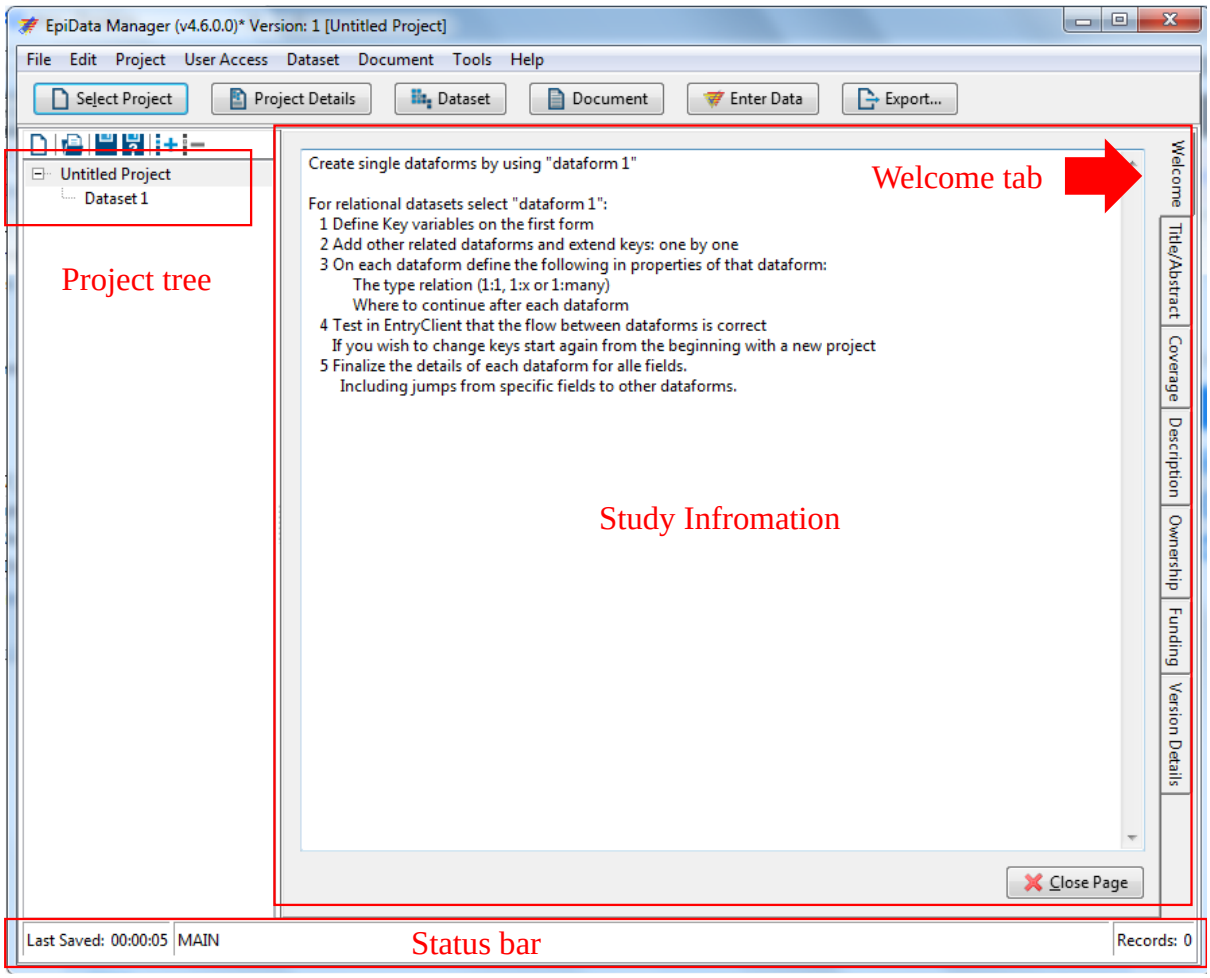


Figure 2.3.1 Project tree, Study Information and status bar

Project Tree lets you navigate through all dataforms under the tree structure. You can easily switch the main window from the project's **Study Information** to other dataforms by pointing and clicking there on the tree structure. As you can see on figure 2.3.1, the name of the project is still "Untitled Project". To change the name, you can edit by **double-clicking** on it. We will do that later.

Study Information is essentially ***meta-data*** or ***data about data***. The full set of Study Information is known as the *Dublin Core Collection*. Read more about it here [https://www.dublincore.org/]. In EpiData, there are seven categories of meta-data or so-called **tabs** which include "Welcome" tab. You can close welcome tab by clicking on "Close Page". It's not that important. The other six tabs are

1. Title/Abstract
2. Coverage
3. Description

4. Ownership
5. Funding, and
6. Version Details.

Table 2.3.1 summarize these information.

Tab	Information	Description
Welcome		Provides a brief introduction on how to create single dataforms (also called dataset) and relational dataforms. This disappears when you click the "**Close Page**" button located at the bottom-right of the screen.
Title/Abstract	Title and Abstract	Immediately after the title is changed, the name of the project under "**Project Tree**" gets changed accordingly. The abstract should contain a short summary of the project.
Coverage	Geographical	Study location
	Language	Specifies the current language used in the project. This is particularly useful as some projects can be conducted through multicenter collaboration. However, this field is disabled in current version.
	Date time coverage	Study period in *dd/mm/yyyy* format.
	Population	Study population, samples and sampling procedures, if any.
	Units of observation	Study units, tools and measurements.
Description	Keywords	Any specific key words.
	Purpose	Specify rationale, research questions, aims & objectives and implications of the study.
	Citations	Allows citations to be added.
	Design	Study design
Ownership	Organization/Institute	Names of organizations, institutes, and other partnerships.
	Agency (Short acronym …)	Name of the place where the data is stored, e.g. headquarters.
	Authors/Contributors	Authorships and any acknowledgements of contribution to the project.
	Rights	Any statements of copyrights, disclaimers and credits.
Funding		Any funding statements.
Version Details	Identifier	Study or Project identification number.
	Version	Version number of the project. This information is particularly useful when you continuously develop the project. Default is number "1".

Table 2.3.1 A summary of Study Information

Status bar at the bottom of the screen currently gives three pieces of information: 1) **Last Saved**, 2) **MAIN**, and 3) **Records**. "**Last Saved**" shows the time in hours, minutes and seconds since you last saved the project. "**MAIN**" indicates that the **canvas** is currently selected. We will cover this more at later chapter. "**Records**" shows the number of current data entries in the dataset. Currently we have none since this is a new project.

Let's move on to the dataform and see what happens. Click on the "**Dataset 1**" dataform under the **Project Tree**. The screen on the right changes to a blank screen with small grid layout. We call this a **blank canvas** because later on we will create our data entry form on it. It's like painting on a blank canvas. When you maximize the application window, you may notice a red dashed line on the right edge of the blank canvas. This line indicates the margin of the form when you print. Above the **blank canvas**, there is a set of tools to create data entry fields. We will cover this more in the next chapter.

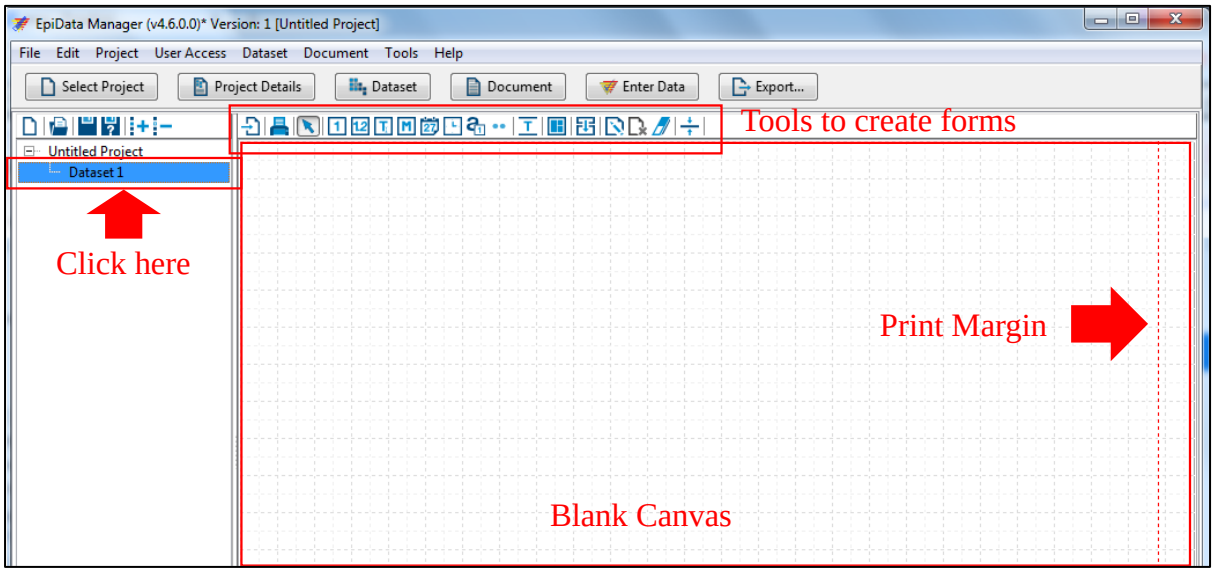


Figure 2.3.2 Blank canvas in dataform

Let's open the "**Title/Abstract**" tab. Change "**Untitled Project**" to "**Form 1**" and press "`Enter`". You may notice that the name under the **Project Tree** also gets changed.

2.4 Saving and Closing the current Project

To save the project file, click "**File**" and then choose "**Save Project As**". Navigate to the folder of your choice. Give file name "**form1**" and then click "**Save**". Alternately, you can use keyboard shortcut "`Shift + Ctrl + S`".

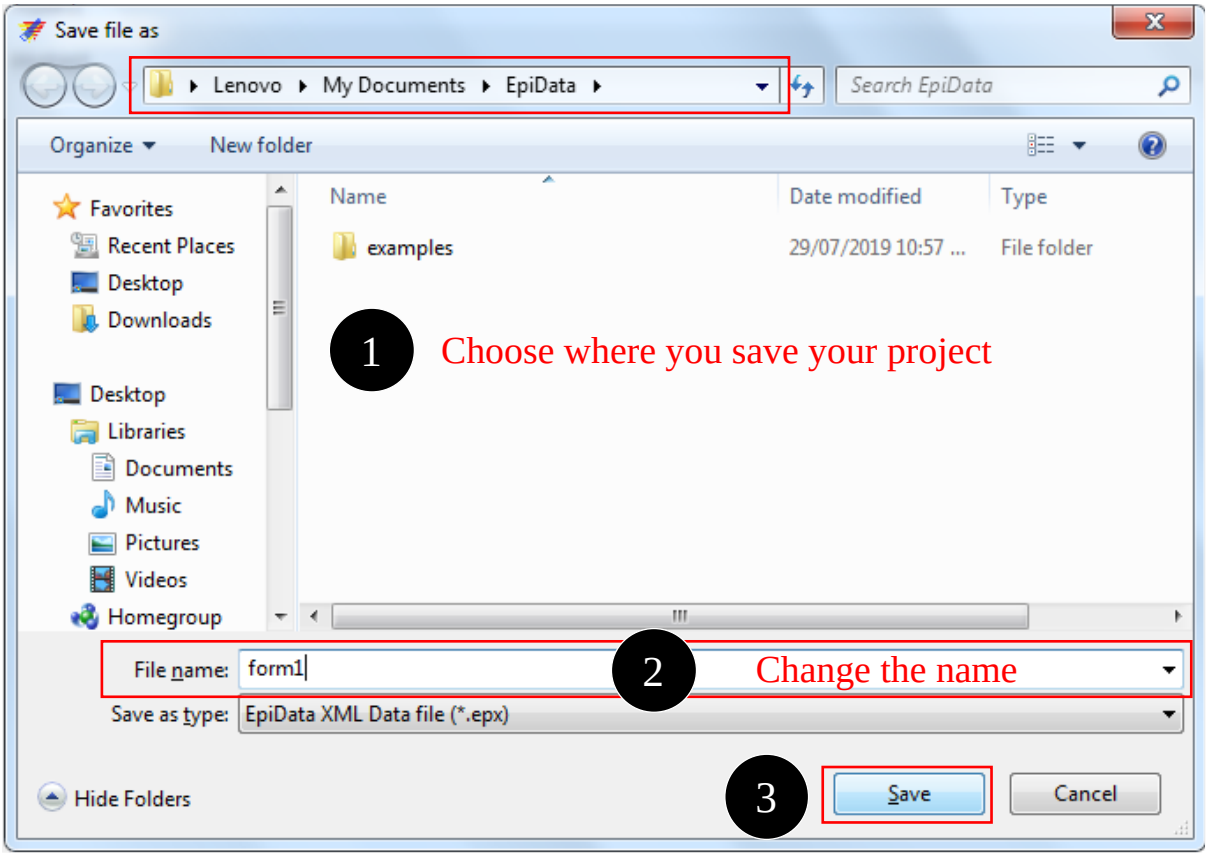


Figure 2.4.1 Saving project

As soon as you save the project, "**Cycle No.: 1**" will appear in the status bar. This means the number of times you save your project. Each time you save the project, this number will increment by 1. "**Cycle No.**" and "**Last Saved**" in the status bar goes hand in hand. Every time the number changes, "**Last Saved**" starts time from zero again.

To close the project, click "**File**" in the menu bar and choose "**Close Project**".

2.5 Opening existing projects

To open an existing project, click "**File**" in the menu bar and choose "**Open Project**" or "**Open Recent**" for any previously opened project. Alternately, press "`Ctrl + O`" to open a project. Or press "`Ctrl + Shift + 1`" for the recently opened project. See the annexure for more keyboard shortcuts.

2.6 Example project: Form 1 in Tuberculosis Programme

Let's introduce to "Form 1" used in Tuberculosis (TB) control programme. This form is recommended by World Health Organization (WHO) as standard recording tool. Please refer to the book "**The revised TB recording and reporting forms Version 2006**" released by WHO. [https://www.who.int/tb/publications/tb_r_and_r_forms_2006/en/]

This example is about tuberculosis for the purpose of demonstrating a step-by-step tutorial while showcasing the features of EpiData Manger and EntryClient.

Tuberculosis Programme Form 1

Request for Sputum Smear Microscopy Examination

The completed form with results should be sent promptly by laboratory to the referring facility

Referring facility[1] _____ Date _____

Name of patient _____ Age _____ Sex: ☐ M ☐ F

Complete address _____

_____

Reason for sputum smear microscopy examination:

☐ Diagnosis

OR ☐ Follow-up Number of month of treatment: _____ BMU TB Register No.[2] _____

Name and signature of person requesting examination _____

1. Including all public and private health facility/providers

2. Be sure to enter the patient's BMU TB Register No. for follow-up of patients on chemotherapy

RESULTS (to be completed in the laboratory)

Laboratory Serial No. _____

Date collected[3]	Sputum Specimen	Visual appearance[4]	RESULTS				
			NEG	(1-9)	(+)	(++)	(+++)
	1						
	2						
	3						

3. To be completed by the person collecting the sputum

4. Blood-stained, muco-purulent, saliva

Examined by _____

Date _____ Signature _____

Figure 2.6.1 Form 1 to request for Sputum Smear Microscopy Examination

Task 2.6. Fill in the **Study Information** of current project "**Form 1**" and save the project as "form1.epx".

Tab	Information	Description
Title/Abstract	Title	Form 1
	Abstract	Aims to demonstrate the functionality and features of the EpiData Manager and EntryClient using Form 1 from Tuberulosis Programme
Coverage	Geographical	Country X
	Language	English [**currently this field is not enabled**]
	Date time coverage	1st January 2018 – 31st December 2018
	Population	Persons who request for sputum smear microscopy examination
	Units of observation	person
Description	Keywords	Form 1, tuberculosis, sputum smear
	Purpose	To demonstrate the functionality and features of the EpiData Manager and EntryClient
	Citations	The revised TB recording and reporting forms Version 2006, World Health Organization, WHO reference number: WHO/HTM/TB/2006.373
	Design	Cross-sectional
Ownership	Organization/Institute	National Tuberculosis Programme
	Agency (Short acronym …)	NTP
	Authors/Contributors	John Smith
	Rights	TP owns the data: access and any types of usage will be required to submit formal request for official permission. Copyrighted by NTP.
Funding		Funded by TP & WHO
Version	Identifier	form1
Details	Version	1

Table 2.6.1 Study Information of the project "**Form 1**"

Chapter 3. Creating a codebook

At this stage, many people usually dive into the process of creating dataform in EpiData. But I highly recommend to first create a codebook. So, what is a codebook?

"A codebook is a type of document used for gathering and storing codes." - Wikipedia

A codebook is basically a document with a list of codes and description or instruction of data entry in research. Sometimes, it is called data dictionary. This is an essential first step to data documentation to the whole data collection process. It should be comprehensive and at least contain instructions on how to proceed in all possible data entry scenarios. For example, what type of data will be entered for the entry field "**age**". Will we enter it as a number representing the age in years or as a category representing certain groups of age? If numeric, how large can the value be? A hundred or a thousand? How many decimal places will be allowed? All these questions will be clear once you develop a codebook.

It can also anticipate some questions that a data entry staff might ask, and subsequent instructions can be provided beforehand. Hence, it should be a comprehensive guideline providing all the necessary details of all variables. This codebook should be made available to all data staffs involved with data collection who should be trained how to use it and make reference to it in case of any queries.

3.1 Characteristics of a codebook

Generally, the following seven characteristics of variables should include in your codebook.

Meta-data	Description
Name	Name of entry field
Label	Description of entry field
Type	Data type
Length	Length of entry field
Range	minimum and maximum values allowed for numeric and date data.
Value Labels	Values and labels assigned for levels of the category Special number such as 9 or 99 can also be used to represent missing data.
Comments	Any instructions for data entry staffs. Adding Notes: or stating calculated variables / deriving variables.

Table 3.1.1 Codebook

Name

Every variable in the project should have a unique name of its own. However, there is no one standard rule for assigning names to variables. Variations exist from different computer programming languages and different organization like Google, Facebook or Apple. However, there are some generic rules for naming convention.

1. **Start the name with an alphabet**.

   Within the variable's name, you can use all alphabets and numerics as well as an underscore "_" character. For example, "`weight1`" is acceptable where "`1weight`" is not.

2. **Use a single word**.

   It means that name should not contain a space(s) or special characters.

   are not acceptable "`age`" is a simple example. For age at registration, something like and for age at death, can be used.

3. **Use an intuitive name**.

   For example, age at registration for TB can be "`age_reg`" and age at death "`age_death`". Likewise, date of registration could be "`date_reg`" while "`dor`" may not be very readable. However, "`dob`" is a commonly used acronym for date of birth.

4. **Make distinction of different composite of words**.

   For example, "`age_reg`" and "`age_death`" shows that you can combine with an underscore that makes them easier to understand. This style is descriptively called **snake_case**.

   Another style uses an uppercase letter at the start of the second word combination. Examples would be "`ageReg`" or "`ageDeath`". This is called **Camel case** or more descriptively, **camelCase**. These two styles may be better than simply "`agereg`" or "`agedeath`".

5. **KISS: keep it short and simple**.

   Usually aim to keep about 8 – 10 characters per name.

Label

Labels are straightforward. But keep "**KISS**" principle in mind.

Type

In EpiData, there are three basic types of data:1) String, 2) Number and 3) Date. Strings can be just a short text or long string called "**memo**". Numbers can be an integer, floating

number (number with decimals), auto-incremental numbers and times. Dates are usually in "dd/mm/yyyy" format, but other types can be offered.

Two other special types are **Boolean** (1 as Yes and 0 as No) and **UPPERCASE STRING**.

Sometimes numbers are used to represent categorical data. The reason is that **humans make less error when they type less, and they make less error when they type a number rather than text**. Example, sex of a subject is a categorical data and usually include male and female. Let's take a moment and think here. We can create a field of string to input either "male" or "female" or we can just enter "M" or "F". However, keying "1" or "2" is much easier. The numbers "1" and "2" does not bring any mathematical sense here but represent being male or female.

For dates, a valid date should be entered, meaning that if you put "`30/02/9999`", this will not be accepted by the EpiData.

Type can provide very basic check for data validation. Example, you cannot input a string into a numeric field.

Range

Range is also another type of built-in check to reduce the data entry error. Example, you are entering data of adult subjects aged > 18 years old. If a range is provided, entering values less than 18 would give a warning or an error while data entry.

It is usually used for numerical data, either discrete or continuous. Dates can also be given a range.

Value labels

This comes hand in hand with numbers representing categorical data. In our previous example "sex", the number "1" represents "male" and "2" "female". Unless we provide labels to the value, we will not know which numbers mean which.

Another use of value label is assigning **UNKNOWN** or **MISSING** values.

Notes

As a general rule of data entry, **a value should be entered to every variable**. The reason is that when a value is missing, you don't know whether it is missing in the original record or data entry staff forgets to enter. Missing values should also be pre-defined in the codebook. This will enable the uniformity in data entry process if you are collaborating with several project sites or areas. However, this practice is controversial and open to debate among data managers.

3.2 A Codebook for Form 1

Myanmar National TB Control Programme want to collect data regarding request for sputum examination from laboratories of six major referring facilities (Yangon, Mandalay, Nay Pyi Taw, Taunggyi, Kalaw, Rahkine) between January 2010 and December 2018. Sputum examinations are usually requested for adults (>= 18 years old).

Note: Three letter codes for cities are `YGN` for Yangon, `MDY` for Mandalay, `NPT` for Nay Pyi Taw, `TGG` for Taunggyi, `KLW` for Kalaw and `RHK` for Rahkine.

Task 3.2. Create a codebook using "Form 1" shown in Figure 2.6.1.

SOLUTION:

Name	Label	Length	Type	Range	Value Labels	Notes
facility	Referring facility	3	String	-	YGN = Yangon MDY = Mandalay NPT = Nay Pyi Taw TGG = Taunggyi KLW = Kalaw RHK = Rahkine	-
dateRef	Date of Referral	10	Date	01/01/2010 – 31/12/2018	01/01/1900 – missing values	Enter 01/01/1900 if Missing.
ptName	Name of patient	20	String	-	-	Enter MISSING if Missing.
ptAge	Age	2	Integer	18 – 90	99 – Missing value	Enter 99 if Missing.
ptSex	Sex	1	Integer	-	1 – Male 2 – Female 9 – Missing value	-
ptAddress	Compete address	-	Memo	-	-	Enter MISSING if Missing.
reason	Reason for examination	1	Integer	-	0 – Diagnosis 1 – month 1 2 – month 2 3 – month 3 4 – month 4 5 – month 5 6 – month 6 7 – month 7 8 – month 8 9 – Missing value	*If 0, enter 8888 in next field and skip.* *If 9, enter 9999 in next field and skip.*
regNum	BMU TB registration number	4	Integer	1 - 9000	8888 – "Not Applicable" 9999 – "Missing"	*If 0, this should be 8888.* *If 9, this should be 9999.*

Table 3.1.2 Codebook for "**Request for Sputum Smear Microscopy Examination**" of **Form 1**

Name	Label	Length	Type	Range	Value Labels	Notes
serNum	Lab Serial Number	4	Integer	1 - 9999	-	There should not be missing value.
date1	Date of specimen 1 collected	10	Date	01/01/2010 – 31/12/2018	01/01/1900 – missing values	-
vis1	Visual Appearance of specimen 1	1	Integer	-	1 = blood-stained 2 = muco-purulent 3 = saliva 9 = Missing	-
res1	Result of specimen 1	1	Integer	-	0 = Neg 1 = 1+ 2 = 2+ 3 = 3+ 4 = (1-9) 9 = Missing	-
date2	Date of specimen 2 collected	10	Date	01/01/2010 – 31/12/2018	01/01/1900 – missing values	-
vis2	Visual Appearance of specimen 2	1	Integer	-	1 = blood-stained 2 = muco-purulent 3 = saliva 9 = Missing	-
res2	Result of specimen 2	1	Integer	-	0 = Neg 1 = 1+ 2 = 2+ 3 = 3+ 4 = (1-9) 9 = Missing	-
date3	Date of specimen 3 collected	10	Date	01/01/2010 – 31/12/2018	01/01/1900 – missing values	-
vis3	Visual Appearance of specimen 3	1	Integer	-	1 = blood-stained 2 = muco-purulent	-

					3 = saliva 9 = Missing	
res3	Result of specimen 3	1	Integer	-	0 = Neg 1 = 1+ 2 = 2+ 3 = 3+ 4 = (1-9) 9 = Missing	-

Table 3.1.2 Codebook for "**Request for Sputum Smear Microscopy Examination**" of **Form 1**

Additional Notes

If an entry field has sub-categories, it should be specified as number type, which in this case has no mathematical meaning. The integer codes just correspond to the labels that are defined. The reason for not entering text is that **humans make less error when they type less, and they make less error when they type a number rather than text**. For example, we could define someone's sex as text and then type "male" or "female", or we could also enter "M" or "F" for simplicity, but ideally, we should define integer codes and enter 1 or 2 instead.

Variables which contain a limited number of known categories such as sex (male, female) and marital status (single, married, separated, divorced, windowed) should be defined as numbers and assigned the appropriate integer codes and corresponding labels. Variables which have a larger number of known categories such as place of birth, or which have an unknown number of categories, such as reason for not visiting a doctor, should be defined as text. There are exceptions to these general rules, but these are beyond the scope of this book.

The variable "`reason`" (reason for sputum smear microscopy examination) is a good example of giving an intuitive integer code assignment to labels. People can easily remember that 0 means "`Diagnosis`" and 1 means "`follow-up at 1 month`" and so on. Other good examples are results of specimens, "`res1`", "`res2`" and "`res3`".

Chapter 4. Designing a dataform

In our current project, we have a datafrom named "Dataset 1".

To edit the name of the dataform,

- **Right-click** on it and
- Choose "**Dataset Properties**".

Let's change the name to "`dsRequest`" and the label to "`Request for sputum examination`". (Figure 4.1)

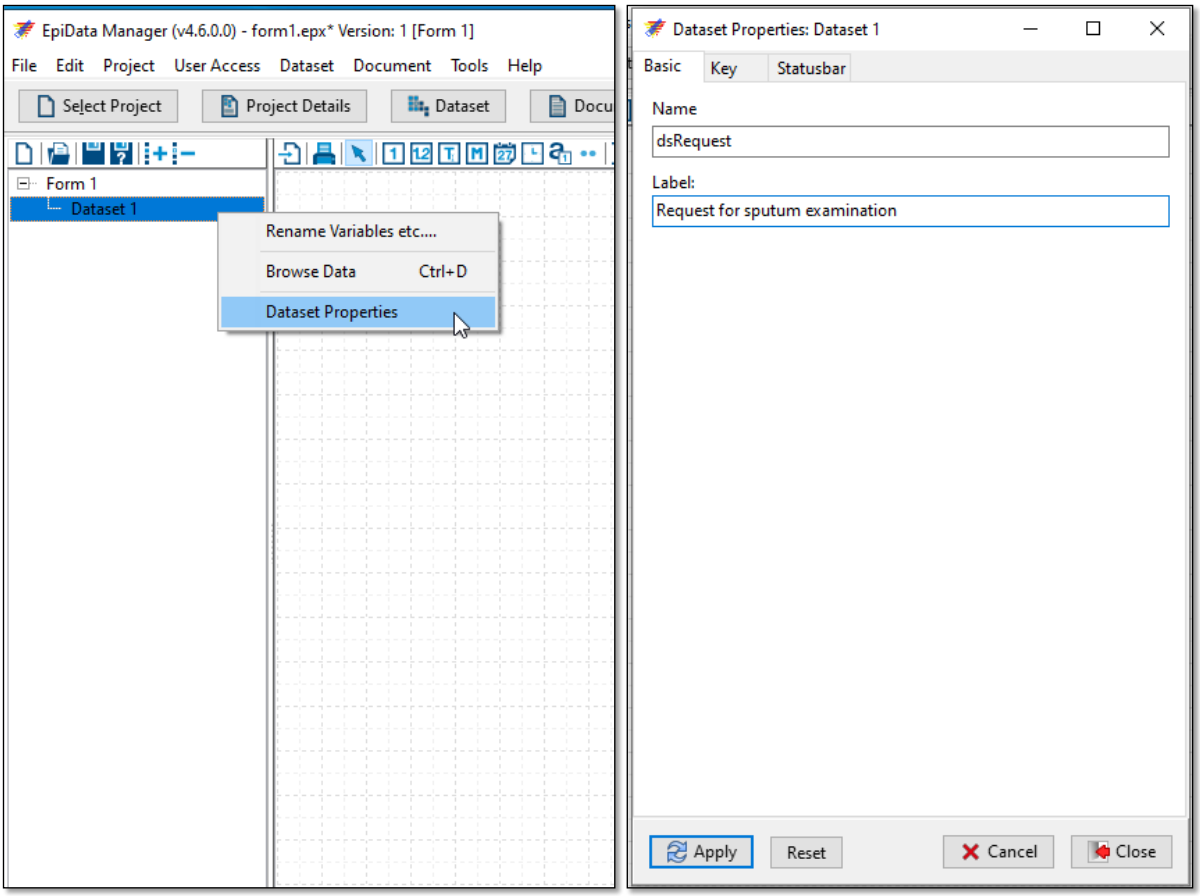


Figure 4.1 Editing the name of dataform

Always remember to save your project periodically as you might never know when your computer will crash!

4.1 Design tools

As we introduced earlier in Chapter 2.3, a toolbar appears at the top of the blank canvas if you click on the dataform. These several tools shown in Figure 4.1.1 are not that many yet powerful enough to create complex dataforms. Their respective functionalities are tabulated in Table 4.1.1.

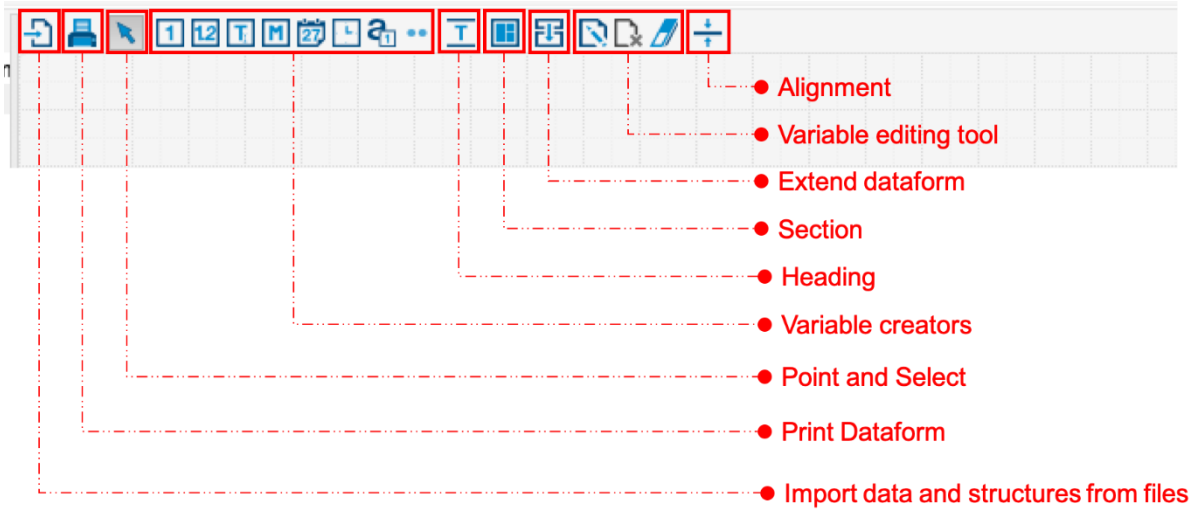


Figure 4.1.1 Design tools to create entry fields for dataforms

Name of the tool	Descriptions
Import data and structures from files	- imports existing data into EpiData Manager - supports a variety of data formats including the old EpiData entry format (.rec), comma-separated values (CSV) and Stata format (before version 13)
Print Dataform	- This is convenient when you want to print your dataform and distribute as paper-based or electronic format as pdf.
Point and select	- Using this, you can point anywhere on the page and select anything on it. - By default, this is selected when you click on dataform under the project tree.
Variable creators	- These are a collective of tools to create different types of data that we introduced in Chapter 3.1. Read it if you are not sure what data types EpiData offers.
Heading	- create headings on dataform.
Section	- group variables together for visual aid and efficient entry.

Extend dataform	-	this extends the height of the dataform.
Variable editing tool	-	edit and delete any or all variables from the page.
Alignment	-	Align variables for visual aid and efficient data entry flow.

Table 4.1.1 Descriptions of Design tools

Three main types of variables under "variable creators" are

1. Number
    a. **Integer**
    b. **Floating point or decimal number**
    c. **Auto-incremental number** – no input required
2. Text
    a. **String** variable – by default, length is 20
    b. **Memo** variable – virtually a very long String variable
    c. **UPPERCASE STRING** variable – all text inputs will be converted into UPPERCASE: same as String variable.
3. Date and time
    a. **Date** – DMY variable in "dd/mm/yyyy" format
    b. Other MDY or YMD formats are also available.
    c. **Time** variable for hours: minutes: second format
    d. **Auto-date and auto-time** – same as date and time variable with specified inputs: No manual input required.

4.2 Adding headings

To add a heading to the dataform

1. Select the dataform.
2. Click on the "**heading**" tool from the toolbar.
3. Move your cursor to the canvas and click.
4. Change the label.
5. Click "Apply" and then "Close".

Let's now add our first heading to the dataform "`dsRequest`". (Figure 4.2.1)

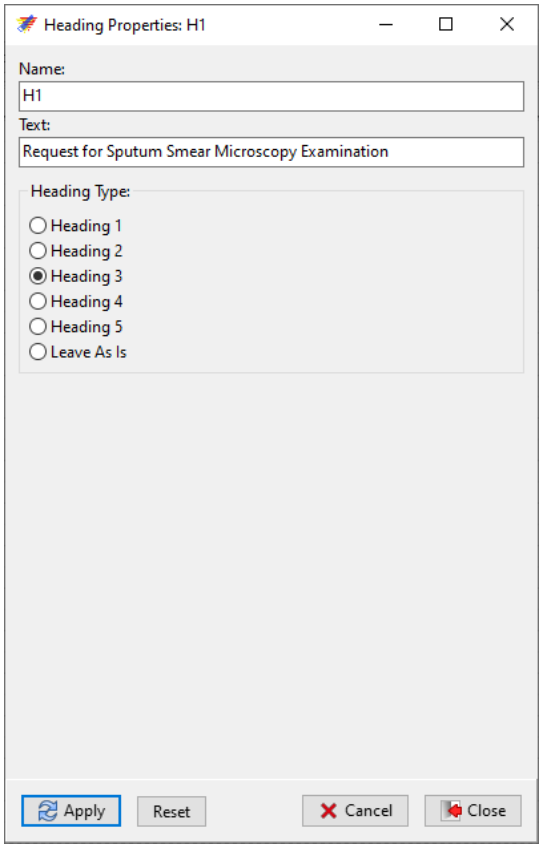
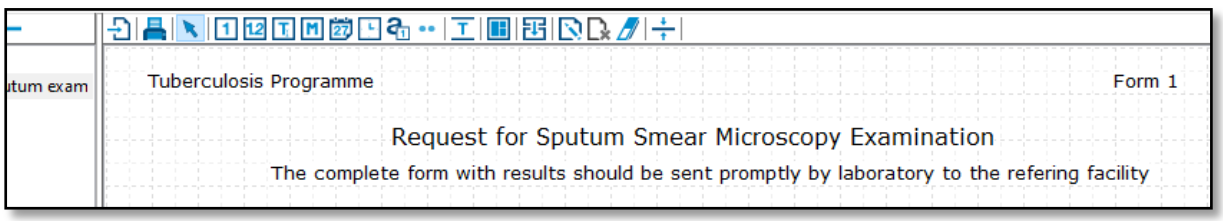


Figure 4.2.1 Adding heading to the dataform

Note:

As you can see the heading properties in Figure 3.4, EpiData provides several levels of font size starting from Heading 1 (the biggest) to Heading 5 (the smallest). Or you can "**Leave As Is**".

Task 4.1.1. Add three more headings to the dataform "`dsRequest`" as shown in Figure 4.1.3.



4.3 Adding variables

String variable

Let's recall our codebook here. The first variable in our "**Form 1**" is **referring facility**. This is a string type of length 3 for entering 3 letter codes. So, we are going to select "**New String Variable**" from the toolbar.

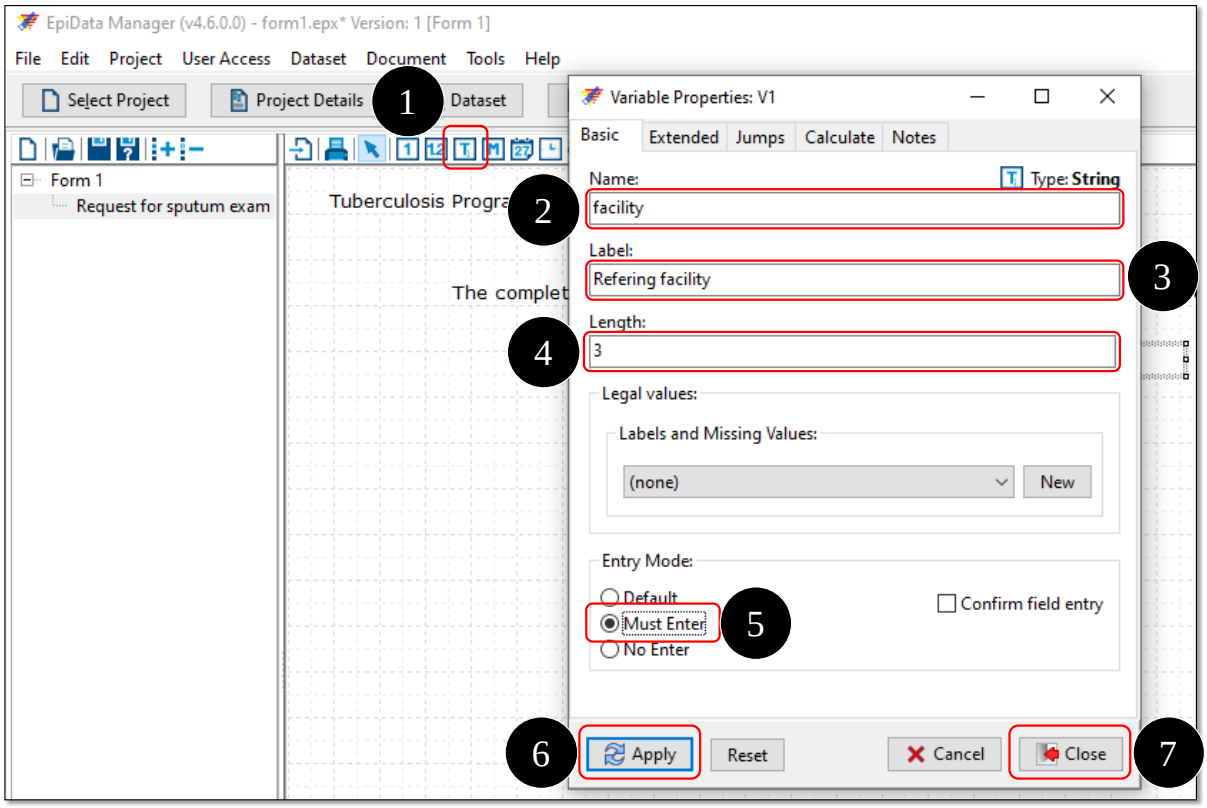


Figure 4.3.1 Variable Properties of "**String Variable**" [*Follow the steps in black circle.*]

Notes:

- "**Legal values**" mean valid inputs. In the case of categorical data, this just means values and value labels. We will step this up next.
- "**Entry mode**" means whether you must input a value or not. In "**Default**" mode, you can either input a value or skip to next variable without giving a value. In "**Must Enter**", you must specify a value and in "**No Enter**" mode, the entry field will not be active and data entry is not possible. This is known as "**no-enter**" field. It is commonly used for derived variables into which values from other fields are feedback.

We have not yet defined the values and value labels for "**facility**". To do this, open the "**Variable Properties**" window by **right-clicking** on the variable and choosing "**Edit**". Or press "**Enter**" key.

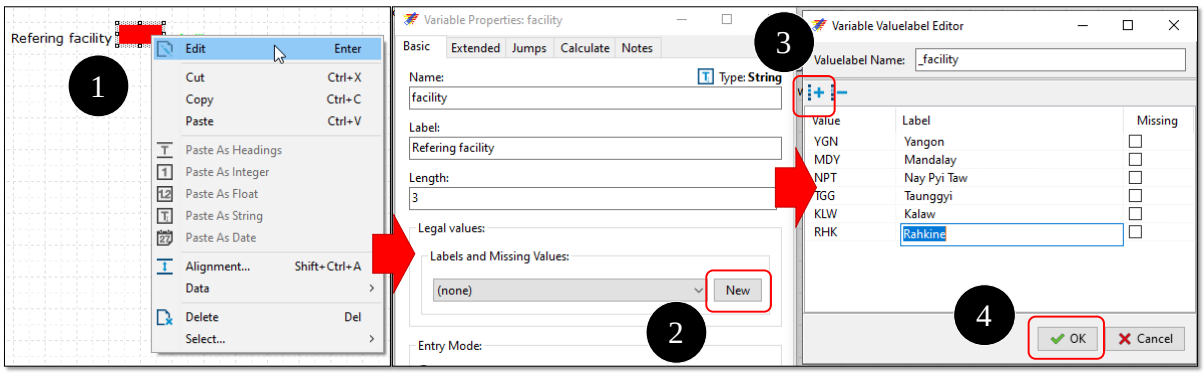


Figure 4.3.2 Adding values and value labels for categorical variable

[*Follow the steps in black circle.*]

Now the last thing to do for categorical variable is to turn on picklist while entering data as shown in Figure 4.3.3. As the name suggests, this shows all available sub-categories to users to choose from. As before, open the "**Variable Properties**" window again. Go to "**Extended**" tab and tick on "**Always show picklist during entry**". (Figure 4.3.4)

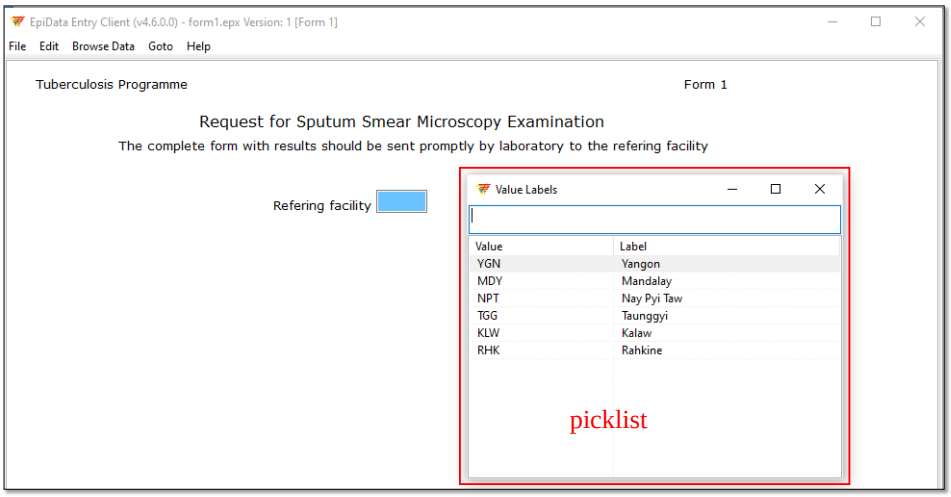


Figure 4.3.3 **Picklist** in action during data entry

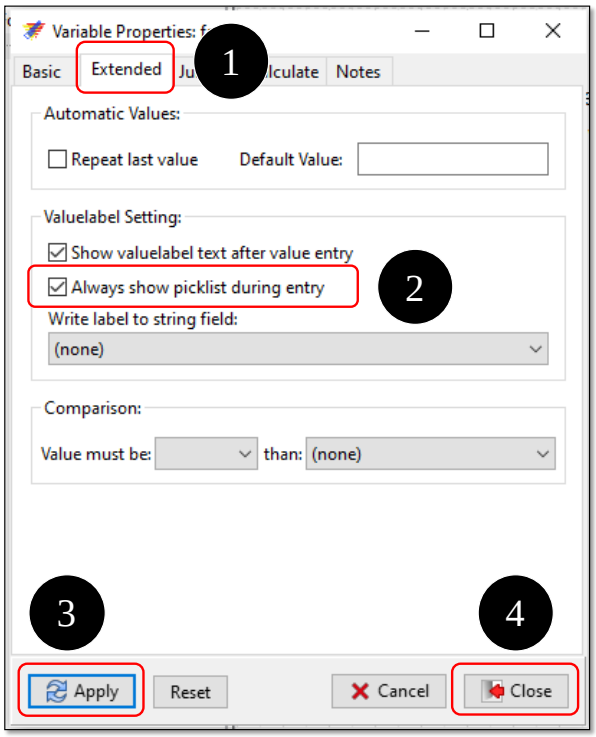


Figure 4.3.4 Turning on "picklist" for data entry

[*Follow the steps in black circle.*]

Notes:

- In future, we will do all these steps at one time. The only additional step is to change "**Valuelabel Name**" in "**Variable Valuelabel Editor**" (the last window in Figure 4.3.2).
- In **Window OS**, when you click on "**Apply**", the window closes. In **Mac OS**, you need to follow all the steps until "**Close**".

DMY variable

Next variable is **date of referral**. This is a DMY type. Even though we specify its length as 10 digits in our codebook, there is no need to specify here. So, we are going to select "**New DMY Variable**" from the toolbar.

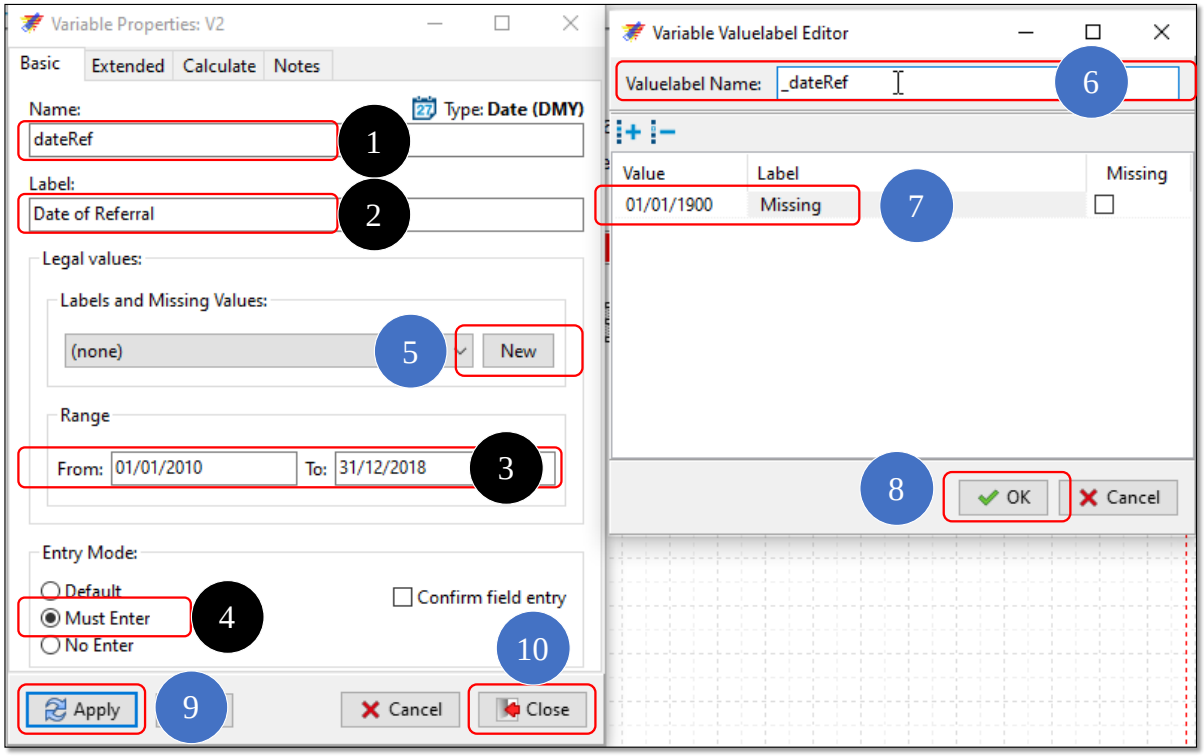


Figure 4.3.5 Adding a DMY variable

[*Follow the steps in black and blue circles.*]

As the last step, open the "**Note**" tab and enter a note "***Enter 01/01/1900 if Missing.***" as shown in Figure 4.3.6.

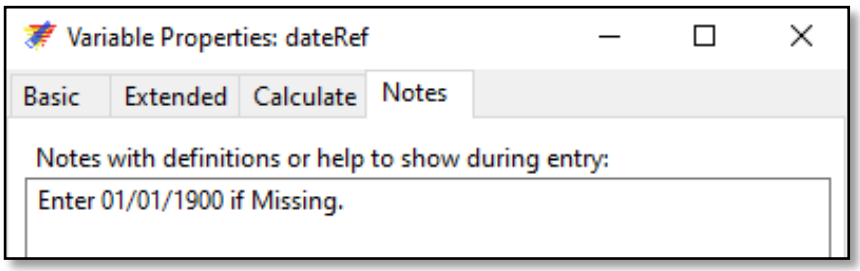


Figure 4.3.6 Adding a note to DMY variable

Task 4.3.1. Create next variable "***Name of patient***" using string variable.

Solution 4.3.1. Adding "Name of patient"

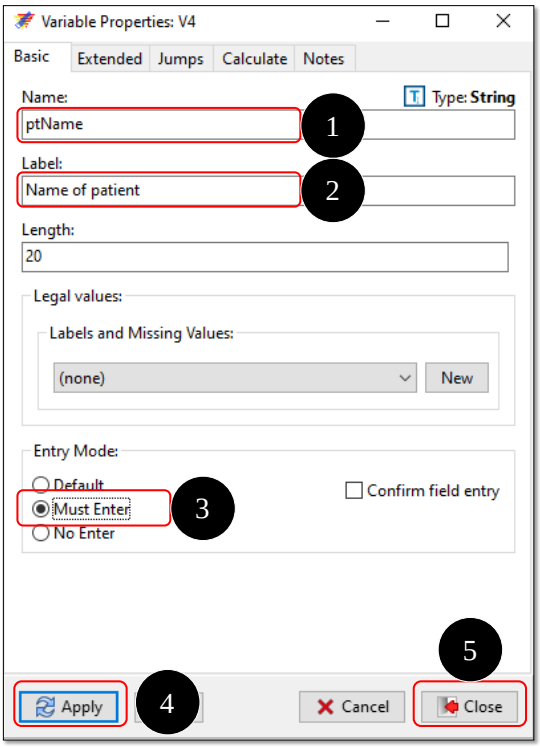


Figure 4.3.7 Adding "Name of patient" to the dataform

Integer variable

Next variable is **age**. This is an integer type of length 2 for entering 3 letter codes. So, we are going to select 1 "**New Integer Variable**" from the toolbar.

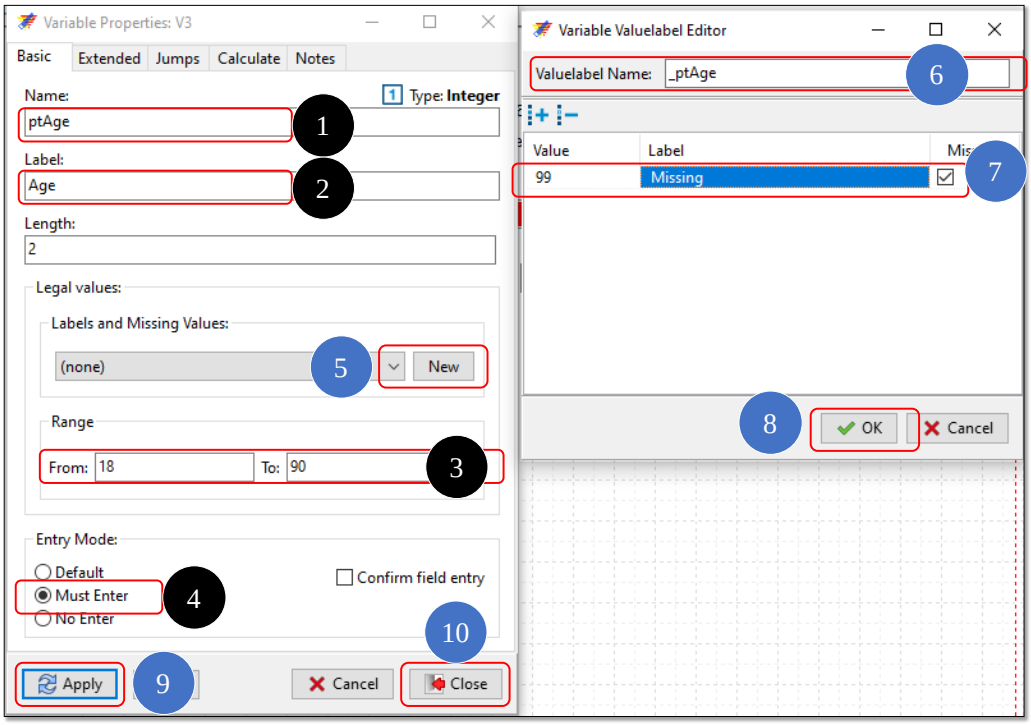


Figure 4.3.8 Adding an Integer variable

As the last step, open the "**Note**" tab and enter a note "***Enter 99 if Missing.***"

Task 4.3.2. Create next variable "***Sex***" using integer variable.

Solution 4.3.2. Adding "Sex" to the dataform

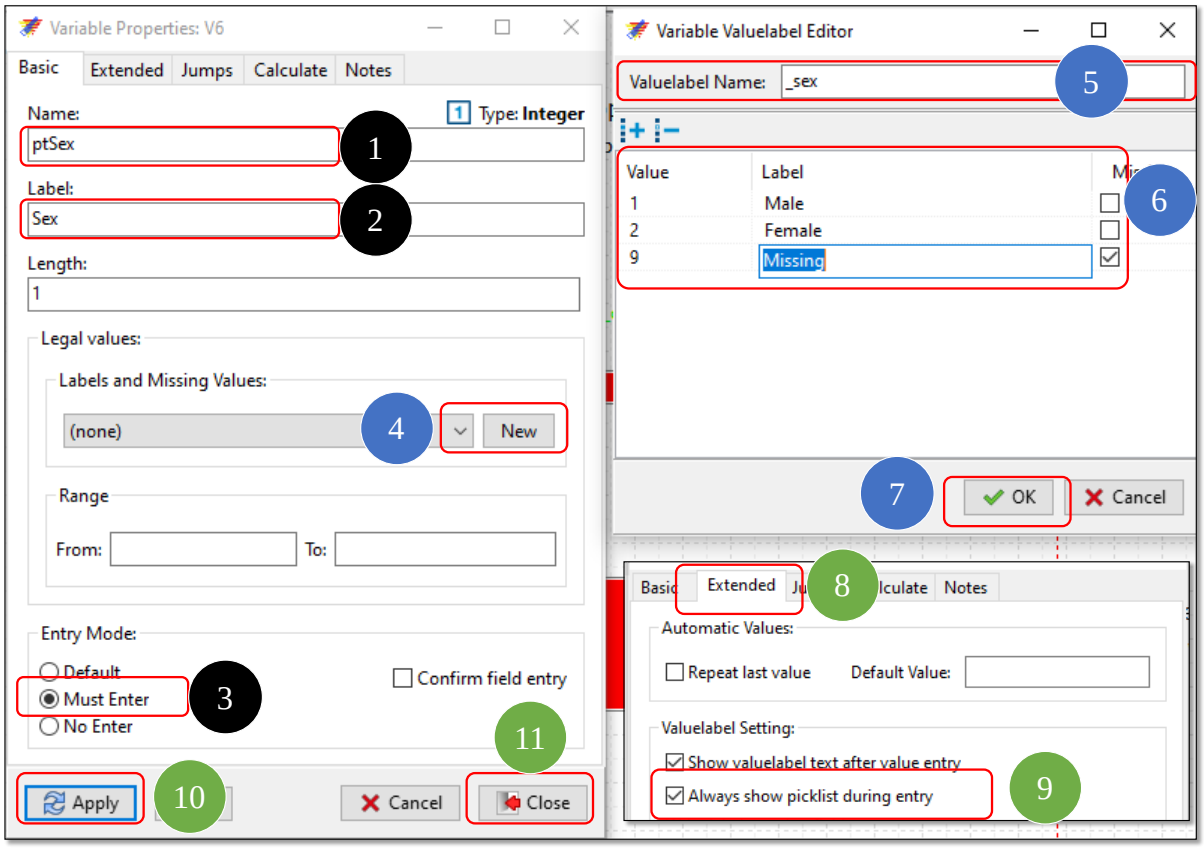


Figure 4.3.9 Adding an Integer variable "Sex" to the dataform

Memo variable

Next variable is **Complete Address**. This is a long string type. So, we are going to select "**New Memo Variable**" from the toolbar.

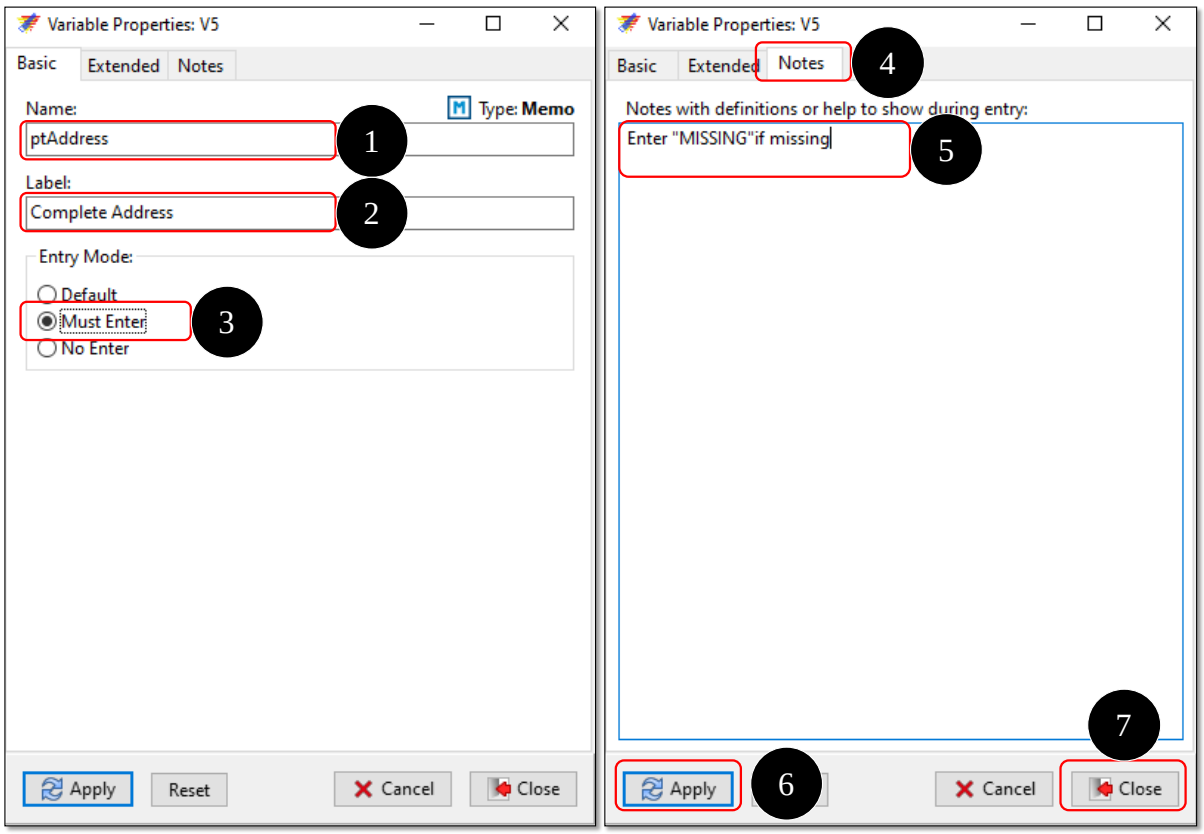


Figure 4.3.10 Adding a Memo variable

Task 4.3.3. Complete the remaining variables from Table 3.1.2.

Note: We will discuss about skipping variables in later chapter.

4.4 Alignment

If you completed task 4.3.3, you should have a dataform similar to Figure 4.4.1. The fields in the figure are displaced and messy. Efficient data entry will not be achieved in this condition. One way to organize our fields is to align them on the right side.

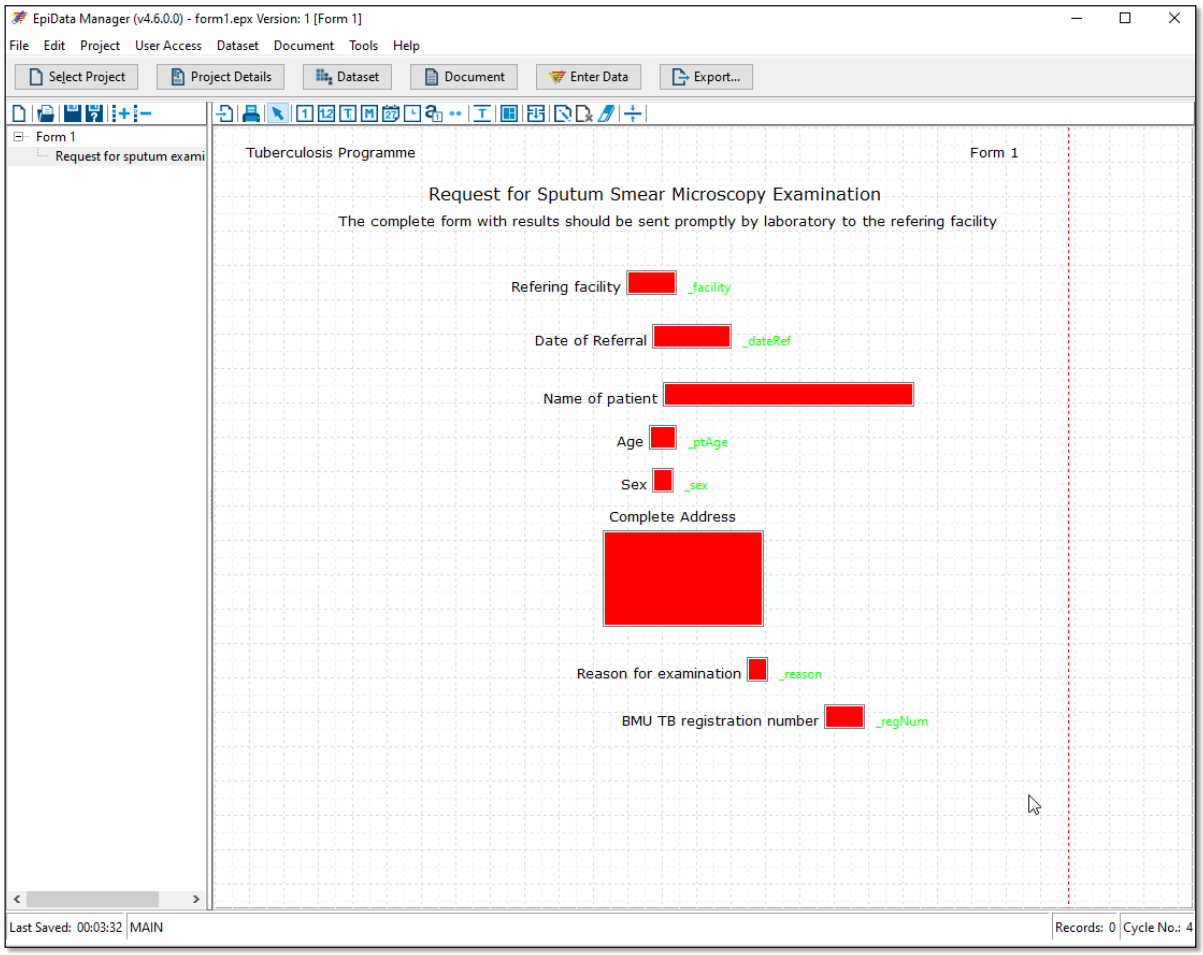


Figure 4.4.1 Misaligned fields in dataform "`dsRequest`"

Alignment tool from toolbar has four main functionalities of alignment. You can explore a bit to know better. Now, we will select all entry fields (not include headings), **right-align** them and keep **vertical fixed distance** as 10 (pixels).

Note:

For window user, do not include "Memo" variable in the alignment because keeping fixed (equal) distance distorts the height of Memo box. (This has not yet been fixed at the time of

writing this book.) Aligning is not rocket science and quite easy as EpiData provides auto-suggested alignment feature (red horizontal and/or vertical lines for alignment).

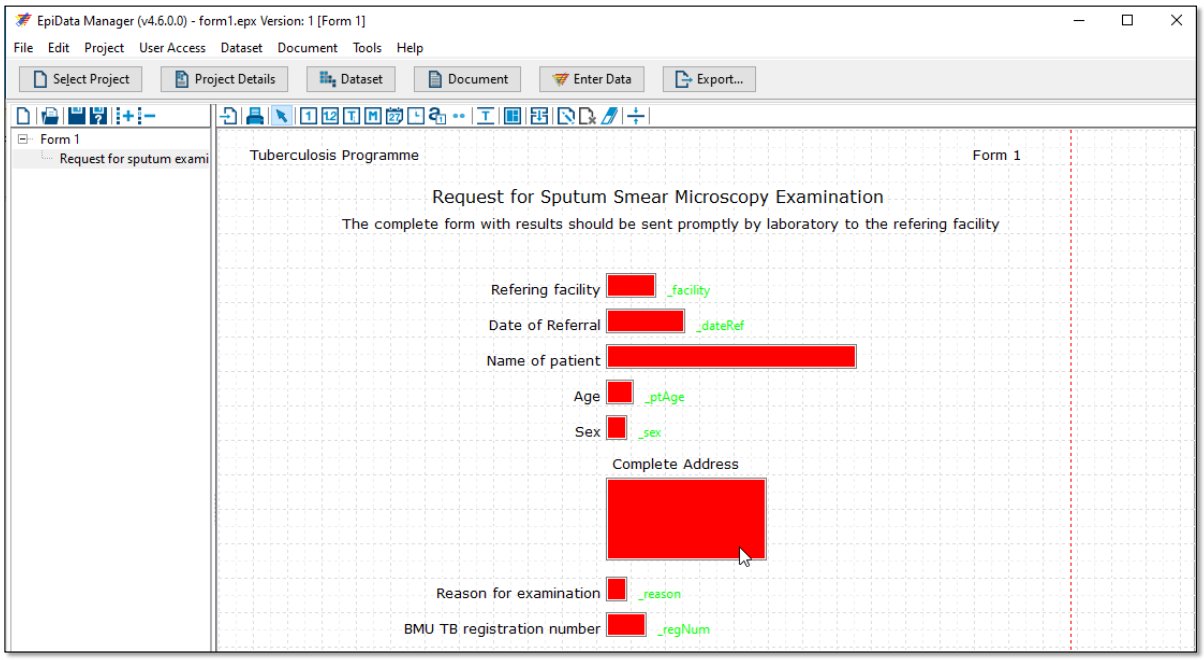


Figure 4.4.1 Fields in dataform "`dsRequest`" after **right alignment of vertical fixed distance** at 10 pixels

4.5 Creating derived fields

Derived fields are variables that do not exist in a data source and are created from one or more existing fields, even across different data sources. (IBM Knowledge Center) In EpiData, this is called as "**Calculated field**". One commonly given example is deriving age from date of birth.

Patient identifier

Keeping the next topic "unique index" in mind, we will add one more variable to the dataform "`dsRequest`" we created earlier. As of now, any variables in our codebook does not provide any uniqueness to the dataform, meaning that there can be duplicated records or data staffs may enter the same record twice and yet our database will still accept them.

To remedy this, we will create a variable to track the patient, namely "`pid`" for **Patient identifier**. This will be integer type with length of 4 digits and no missing value allowed.

Task 4.5.1. Create a variable named "`pid`" with information provided above. Add leading zero from "**Extended**" tab. Place it between date of referral and name of patient.

Note: Leading zero means 0001 for 1 and 0023 for 23.

Solution 4.5.1. Adding variable "`pid`"

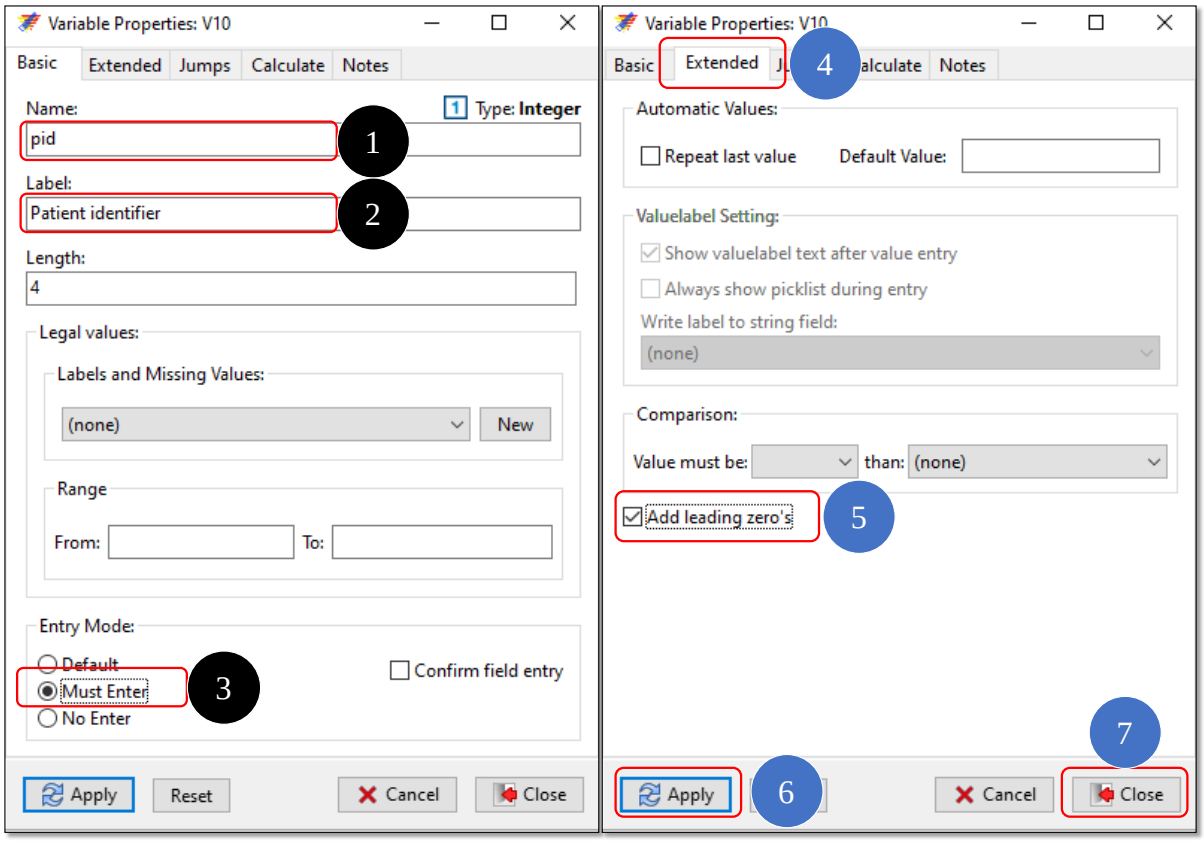


Figure 4.5.1 Adding the variable "`pid`" to the dataform "`dsRequest`"

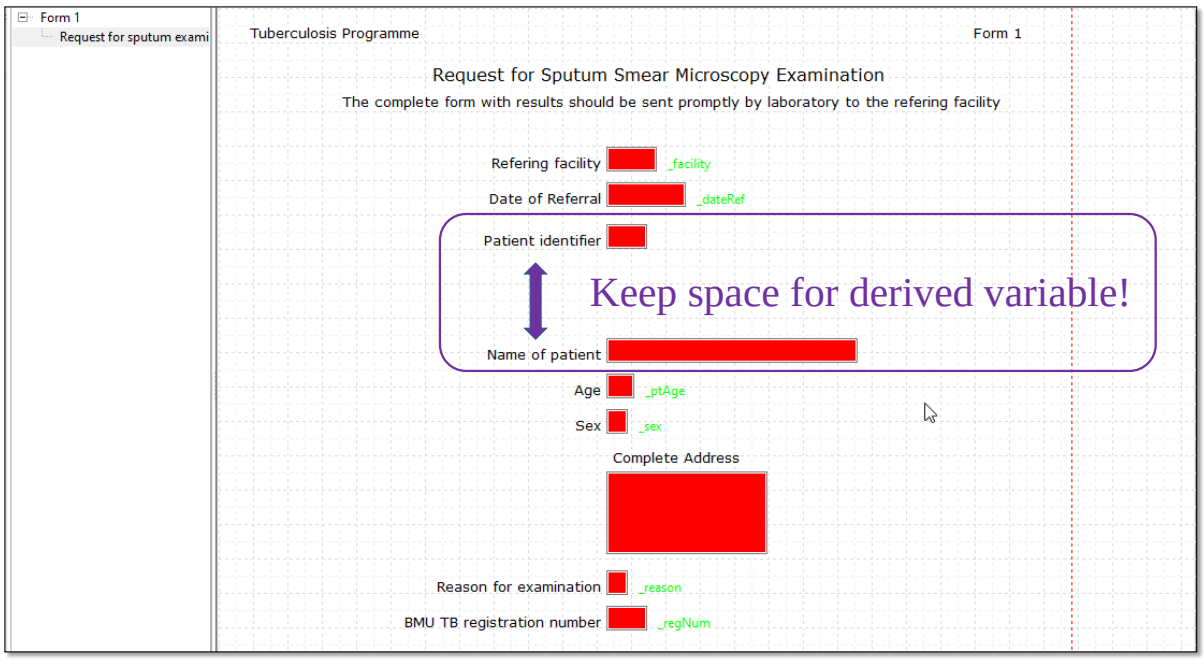


Figure 4.5.2 Alignment to the dataform "`dsRequest`"

Combining into a derived variable

Now we get our patient identifier. The first patient to request for sputum will have `pid` of 0001, second one 0002 and so on. Given that we have six referring facilities, we will have six records of 0001 `pid` after we combine our datasets. This means that our `pid` is not quite unique in the context of six facilities.

If we combine `facility` and `pid` into a new variable `uniqueID`, then that variable will not duplicate anymore. In order to do that, first we will create a "**no-enter**" field to put combined values from the two variables.

```
facility + pid = uniqueID
```

But what data type should it be? A string or an integer?

If you recall the logic that an integer field cannot accept strings or text values, this should be a string field.

What about length?

It is pretty basic mathematic. The variable `facility` has 3 digits and `pid` 4 digits. We want `uniqueID` in the format "`ABC-1234`". Finally, its length is 8 digits.

So, let's add `uniqueID` to our dataform.

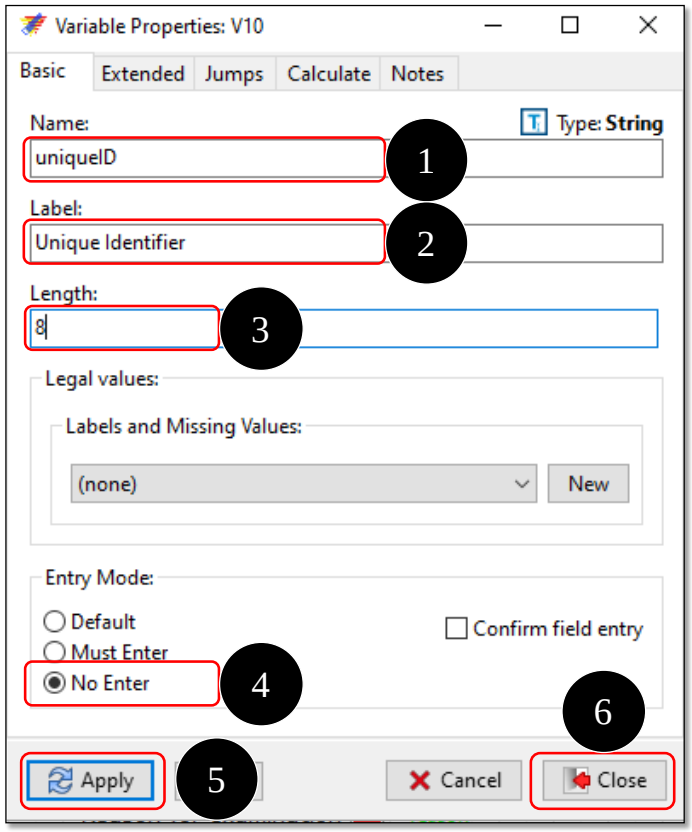


Figure 4.5.3 Adding "`uniqueID`"

Next step in creating derived fields is figuring out where to implement the deriving process. It is quite simple and follows data entry flow. It will be implemented at the last variable before the **NO-ENTER** variable. In our case, it is done at **Patient identifier** as shown in Figure 4.5.4.

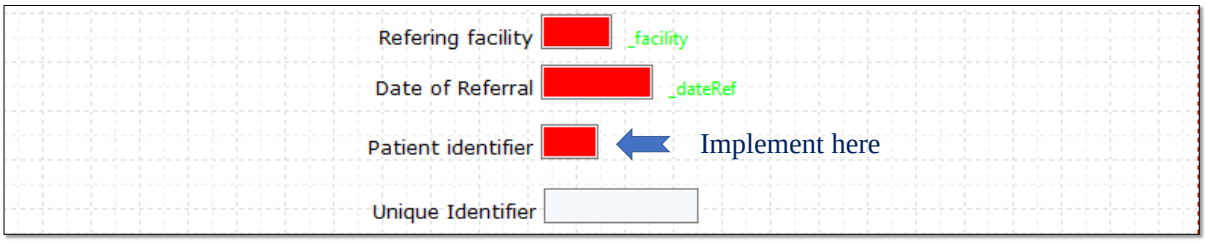


Figure 4.5.4 Figuring out where to implement derivation

Open the "Variable Properties" of **Patient identifier**. If you forget how to do this, recall Chapter 4.3.

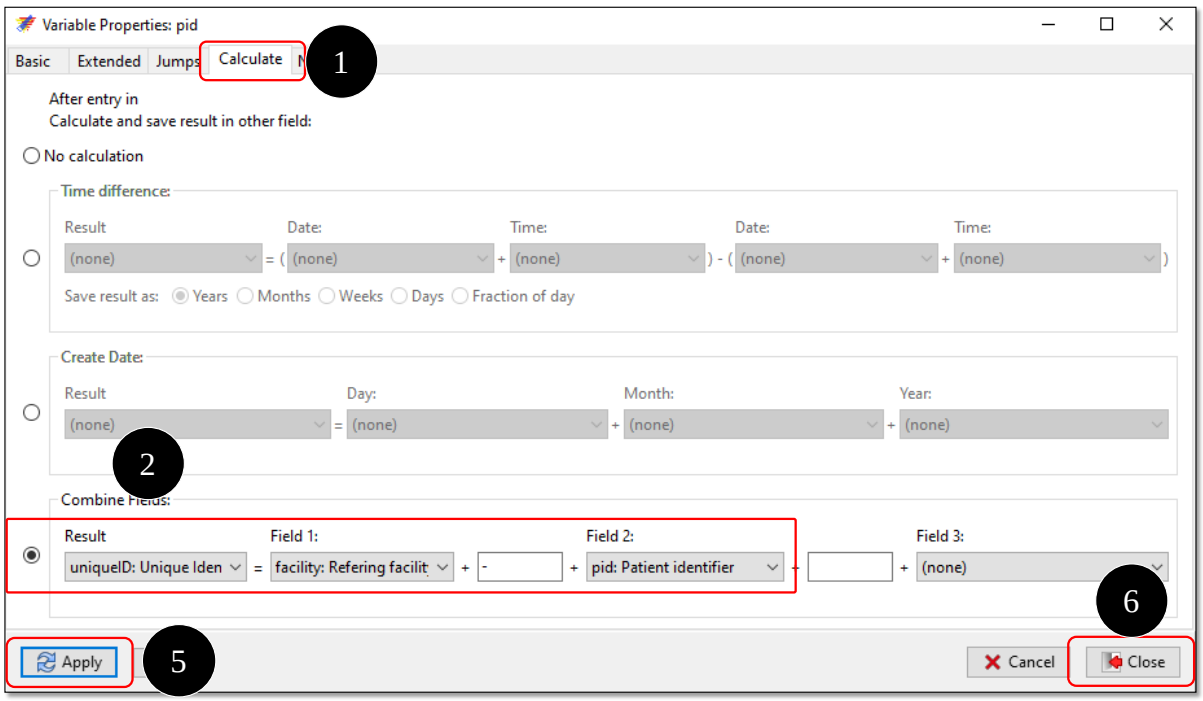


Figure 4.5.4 **Combine Fields** in "**Calculate**" tab for derived field "`uniqueID`"

Now we got our derived field working!

4.6 Unique index

A unique index is an index that enforces the constraint that you cannot have two equal values in the same variable. This is one of the most essential steps in creating a data form, but unfortunately most-often forgotten step. A simple dataform is easy to develop but without a unique index, records with duplicates cannot be distinguished or known.

This serves as a key concept in relational database management where it is used in linking relational child dataforms to their parent form. This is an advanced topic and discussed in Chapter 8.

In order to create unique index, open "`Dataset Properties`" of dataform "`dsRequest`" and follow the steps in Figure 4.6.

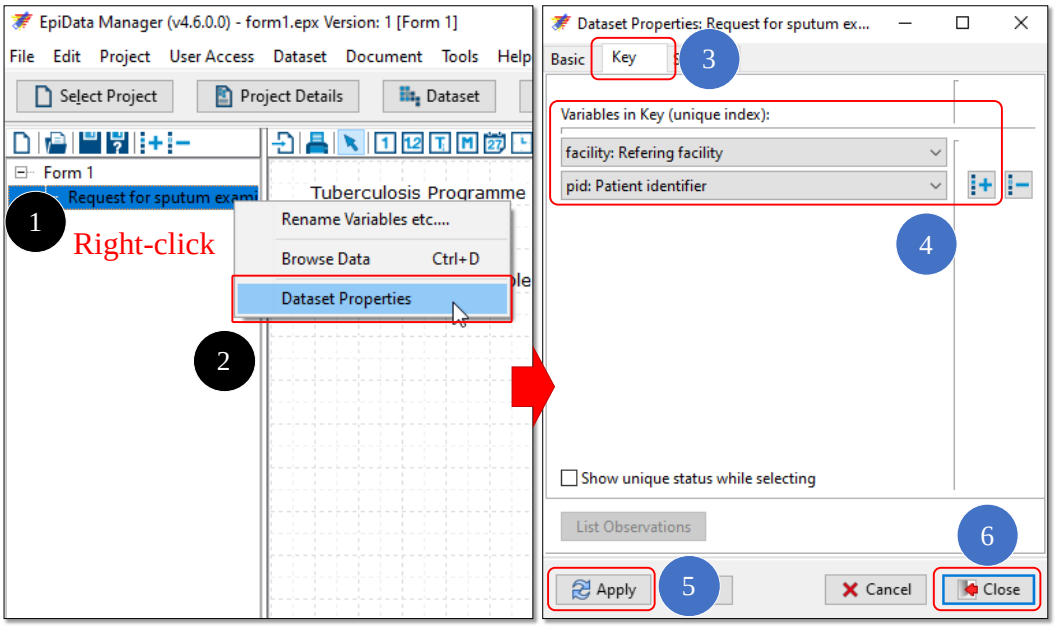


Figure 4.6 Unique index in dataform "`dsRequest`"

Notes

NO-ENTER field cannot be set up as key for unique index. Hence, `facility` and `pid` are used here instead of `uniqueID`.

Each single key field cannot be empty on saving records because of its intrinsic MUST-ENTER property. Hence, after all key fields are keyed in, an implicit search is done by EpiData. This means that users do not need to put any effort to search for any duplicates. When the key index is found, the user can either choose to go to that record or edit values to create a different index value.

4.7 Jumping to next variables

As the name suggests, sometimes we want to jump from one variable and skip several variables that are not applicable (or sometimes missing). A jump needs three information to do its jump. 1) **Jump Value**, 2) **Designated point** (Go To Field), 3) **Reset value** in in-between variables.

Jump Value

This is the desired value that should be valid to the current field.

Designated point (Go To Field)

This can be directed to either one of the four points: 1) **Skip next variable**, 2) **Exit Section** (This closes current project), 3) **Save record** and 4) **specific individual variable** on the dataform.

Reset Value

This is pre-specified value that will be put to all variables between current variable to the designated variable. The value can be left as it is, or it can make use of three types of missing values.

The **first type** is **system missing** value which means that the input value will be represented by the symbol "." in each field. This will be be converted to **blank value** when exporting data.

The **other two types** are **user-defined values**. Recall our codebook at Table 3.1.2 and Chapter 4.3 for creating values and value labels of "**Not Applicable**" and "**Missing**" categories. In "Variable Valuelabel Editor" window, the box in last column "Missing" is ticked. It means user-defined missing values. If two values are checked in this box like in our case "`regNum`", the top one will be "**Second last defined missingvalue**" and the bottom one "**Last defined missingvalue**".

Demonstration

Let's try jumping for "`reason`" and "`regNum`". Recall our codebook at Table 3.1.2. If `reason` has value of 0 (diagnosis) meaning the patient is not diagnosed yet with TB, so there is no way the patient has TB registration number, `regNum`. Hence, `regNum` should be 8888 (Not Applicable). If `reason` is 9 (Missing), `regNum` should be 9999 (Missing) too.

Since there is no variable beyond `regNum`, we will choose **Save Record** for designated point. In `regNum`, 8888 is top row (**Second last defined missingvalue**) and 9999 is the other one (**Last defined missingvalue**).

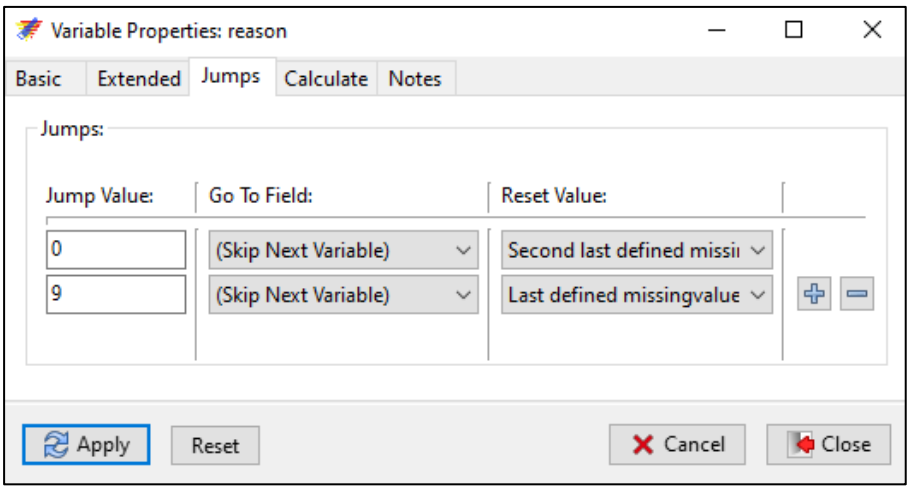


Figure 4.7 Jumping from `reason` to save record

We are now ready to test our dataform in EpiData EntryClient!

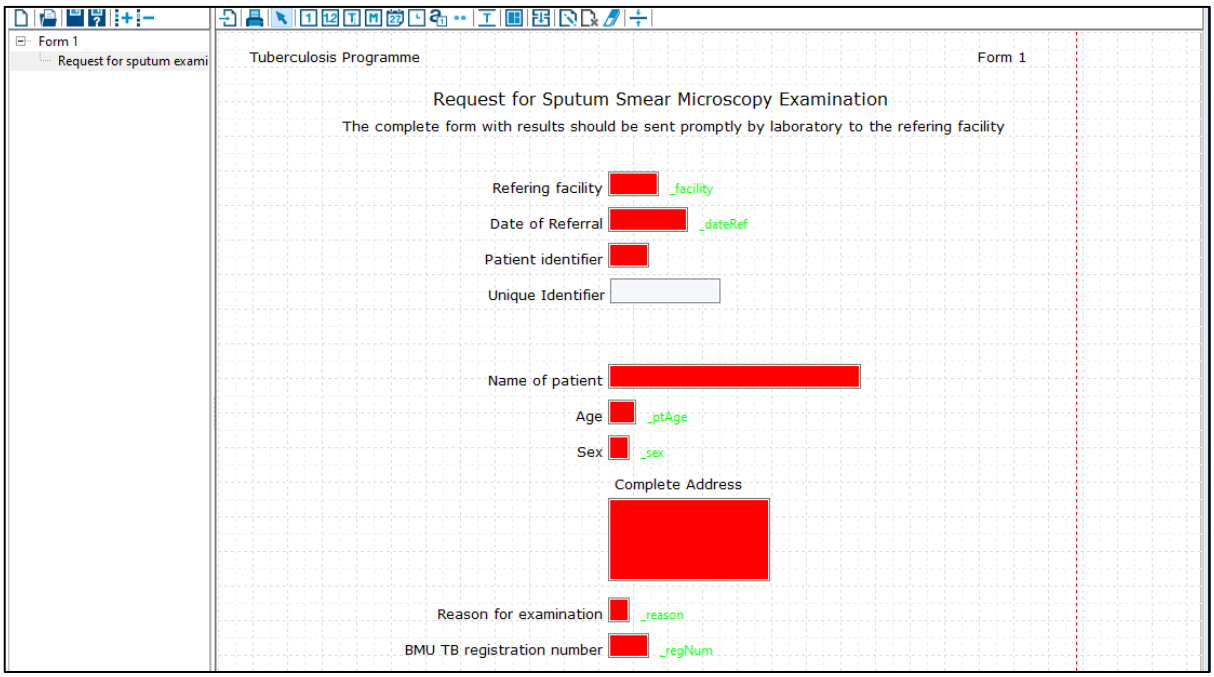


Figure 4.8 **Form 1** ready for data entry in EntryClient

Chapter 5. Getting started with EntryClient

While new projects and corresponding dataforms are created in EpiData *Manager*, the *EntryClient* component of EpiData is specifically for data entry and record management such as deleting records.

5.1 Opening EpiData EntryClient

The interface is quite simple as it is intended solely for data entry and record management. Similar to EpiData Manager, there are menu bar and toolbar as shown in Figure 5.1.

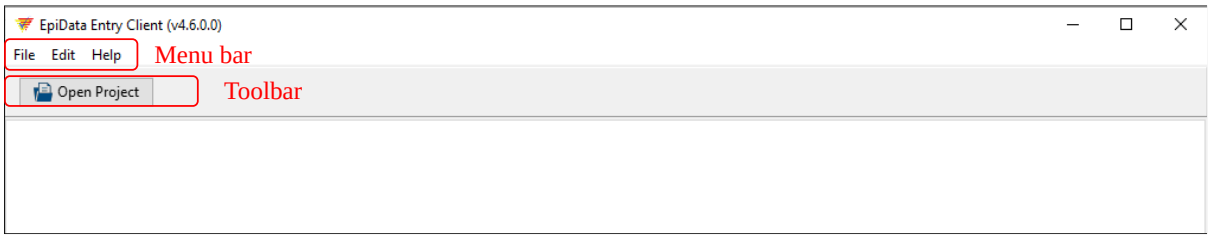


Figure 5.1 Interface of EpiData EntryClient

5.2 Opening a project

To open EpiData project, click "**Select Project**" on toolbar or "**File**" on menu bar, and then choose "**Open Project**". Alternatively, you can use the keyboard shortcut "`Ctrl + O`" on Windows or "⌘ + O" for Mac OS. (See the annexure for a list of shortcut keys)

Let's open our project **Form 1**. As you can see in Figure 5.2, the main form opens along with **Value Labels** window which is introduced as **picklist** in Chapter 4.3. Similar to EpiData Manager, a status bar at the bottom of the window provides **Record Controller** function, **record management** function such as deletion and verification, **key fields**, **field in focus** and **Last Saved** (See Chapter 2.3).

- **Record Controller** function is intuitive as its buttons are familiar to us. `Empty` in the middle just means that there is no record in the dataform at this moment.
- Deletion a record or records will be discussed later.
- **Key fields** indicate which variables are key in the dataform and what their values are.
- Field in focus means which field data entry is currently happening.

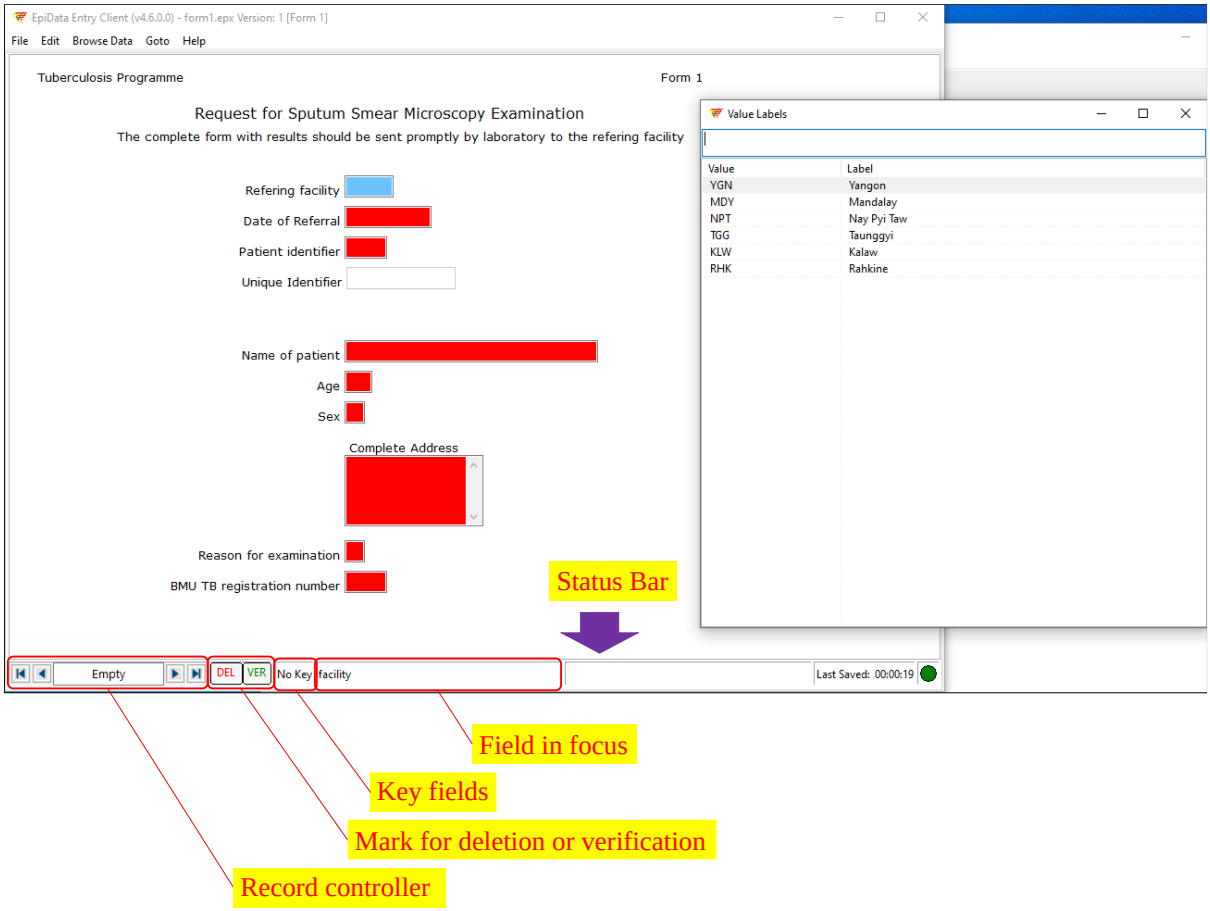


Figure 5.2 **Form 1** project opened in EpiData EntryClient

5.2 Entering values in fields

When **picklist** is shown, you can either use **UP** or **DOWN** arrow key OR enter the desired valid value. In case picklist window is closed, you can press "`Ctrl + F9`" to call it up again.

For numeric fields, the focus will automatically move from one field to another when you enter full digits. For example, patient identifier has four digits. If you enter **2345**, the focus (*cursor*) will move down itself. Otherwise, if you enter **23** which is only two digits, you have to press **Enter** or **Tab** key to move the focus down. The principle is the same for **Date** field.

Although you can use cursor arrow to move the focus, its use will disrupt data entry flow and it is not advisable to do so.

Task 5.2. Enter the following data to Form 1 in EntryClient.

facility	dateRef	pid	ptName	ptAge	ptSex	ptAddress	reason	regNum
YGN	01/01/2010	23	Aung	23	M	Yangon	diagnosis	-
MDY	01/02/2010	43	May	43	F	Mandalay	Month 4	-

5.3 Navigating records

Navigation buttons or **Record controllers** are pretty basic and straightforward. It has **FIRST RECORD**, **PREVIOUS RECORD**, **NEXT RECORD**, and **LAST RECORD**.

Another quick way to navigate your records is through "**Goto Record**" from dropdown menu "**Goto**". However, in this EpiData version, the function does not seem to work. I hope they fix it in their next release.

Another two options are to **List Records** (`Ctrl + L`) OR show **All Data** (`Ctrl + D`). **List Records** show current record and **All Data** displays all records. You can directly double-click on the record you desire in order to open it.

5.4 Printing records

Printing dataform can be handy. In EpiData EntryClient, two printing options are available. The first one is to print the dataform without data (`Shift + Ctrl + P`) and the next one is to print it with data (`Ctrl + P`). Or you can find these options from dropdown menu "**File**".

5.5 Deleting records

EpiData is all about good quality data of which data security is an important aspect. That's why it is tricky to delete a record from EpiData. You cannot just press "Delete" key on your keyboard. There is a special process to it.

In order to delete a record or records, there are two steps.

1. Mark the record(s) you desire to be removed as "**DEL**" DEL in EntryClient and save.
2. Pack the data in Manager.

After you mark the record as **DEL** DEL, just move to next record or previous record. A window will appear asking you to save the modified record as shown in Figure 5.5.1. Save it and close the project in **EntryClient**.

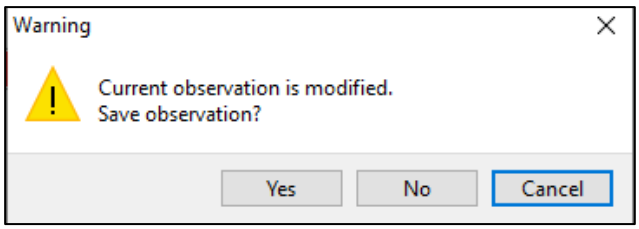


Figure 5.5.1 Window prompt asking to save the modified record

In second step, open **Manager**, go to dropdown menu "**Tools**" and choose "**Pack Datafiles**". Then choose **form.epx** (Change directory if required). A window box with all available dataforms in the project (in this case, **Form 1**) will appear. Tick on the dataform you wish to perform the packing process as shown in Figure 5.5.2. Then click "OK".

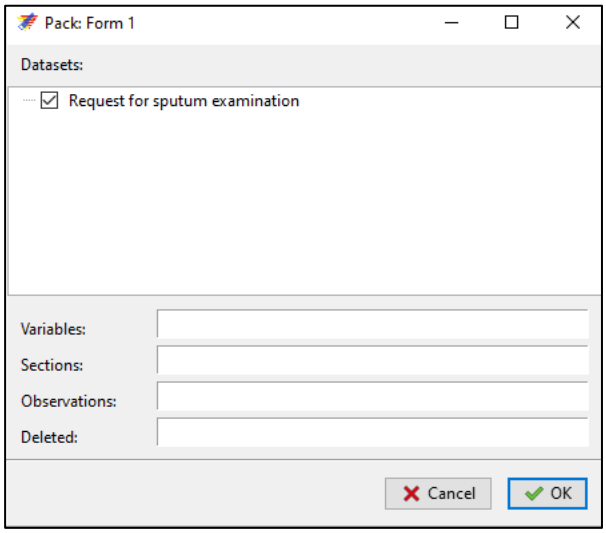


Figure 5.5.2 Window prompt display all available dataforms in the project

As you can imagine, this is tedious and may not be practical if you want to delete hundreds of records.

Note

When a project is opened either in Manager or EntryClient, a temporary `.lock` file is created in the same directory, indicating that the file is in use. So, if you open a project in Manager, you cannot open that project in EntryClient at the same time. EpiData does not allow it.

Task 5.5. Delete the two records we just entered by marking and packing datafiles.

Chapter 6. Double data entry and data management

6.1 Double data entry and validation

Double data entry is not a strange process to us in this digital era. When you register for an account online or when you create a pass lock to your phone, you have to enter your password twice most of the time. This is called two-pass verification or also known as double data entry. It is a data entry data quality control method that have been existed since 20th century when punched cards were popular.

In epidemiological studies or health-related researches, detailed questionnaires with a large number of participants are quite common. Double entry of data coupled with subsequent comparison of data are recommended in such studies. (Reference: Note for Guidance on Good Clinical Practice by European Agency for the Evaluation of Medicinal Products) Such practice definitely provides better quality of data. (Paulsen 2012) However, given that single entry roughly takes up only half of financial and/or human resources, the need for double data entry should be carefully considered for each project.

The following are five steps into double data entry in EpiData, assuming that you have full resources at your disposal.

1. Preparation
2. Double data entry
3. Validation
4. Revision
5. Finalizing

Preparation

Before we start double entry, let's prepare our EpiData file in this regard. To do this, open Manager, go to dropdown menu **Tools** and select **Prepare Double Entry**. Then choose our project `form1.epx`. As you can see from Figure 6.1.1, a window will appear to create a copy of our project. Click **OK** and there you have it, two files for double entry. Even though you can just copy and paste our project, this is a feature of EpiData.

Double data entry

Now we have two files of the same project. Next step is to enter data twice. You may think that one person conducts data entry twice. In fact, this is advisable to use two different persons to do this (of course, if you have resources) because there is a rare chance of doing the same entry errors between two persons. This will improve the quality of your data.

The process goes like this. Let's name these two as A and B.

1. A reads field value out loud to B.
2. B enters the value as he hears.
3. And then B repeats it to A to verbally check or confirm it.

The same process should take place for both A and B's turns. Although it may take up a lot of efforts, this may prevent certain transcription errors such as transposing error or mistyping.

Task 6.1. Let's try this process in our example project **Form 1**. Table 6.1 shows the data of 15 patients requested for sputum smear microscopy examination. Gather two persons (A and B) and enter these data twice using the process described above.

Table 6.1 Datasheet to exercise double data entry and validation

facility	dateRef	pid	ptName	ptAge	ptSex	ptAddress	reason	regNum
NPT	26/09/2017	2282	Maung	48	Male	Nay Pyi Taw	Follow up at month 2	5507
NPT	07/02/2012	6347	Maung	79	Male	Nay Pyi Taw	Diagnosis	-
YGN	30/10/2016	5673	San	53	Male	Yangon	Follow up at month 6	3342
KLW	05/02/2011	3307	Aung	45	Male	Kalaw	Diagnosis	-
MDY	08/03/2011	1859	San	34	Female	Mandalay	Diagnosis	-
NPT	24/10/2017	8646	San	49	Male	Nay Pyi Taw	Follow up at month 6	1664
YGN	26/10/2015	7478	Linn	42	Female	-	Follow up at month 3	4278
KLW	10/04/2017	2480	Aung	33	Female	Kalaw	Diagnosis	-
MDY	14/01/2014	8740	Myo	60	Male	Mandalay	Follow up at month 1	-
RHK	09/01/2012	4064	Aung	49	Female	Rahkhine	Follow up at month 2	145
YGN	-	4618	San	52	Female	Yangon	Follow up at month 3	3543
MDY	04/10/2012	4200	Linn	65	Male	Mandalay	-	540
RHK	20/11/2016	5630	Minn	31	Female	Rahkhine	Follow up at month 1	3845
TGG	08/03/2014	4812	San	60	Female	Taunggyi	Follow up at month 4	5431
RHK	04/03/2018	808	Maung	34	Female	Rahkhine	Diagnosis	734

Request for sputum examination (15)

Show Values / Labels | Sort by Key | Show All/Key | Show All/Filtered Records

Obs. ...	facility	dateRef	pid	uniquelD	ptName	ptAge	ptSex	ptAddress	reason	regNum
1	NPT	26/09/2017	2282	NPT-2282	Maung	48	1	Nay Pyi Taw	2	5507
2	NPT	07/02/2012	6347	NPT-6347	Maung	79	1	Nay Pyi Taw	0	8888
3	YGN	30/10/2016	5673	YGN-5673	San	53	1	Yangon	6	3342
4	KLW	05/02/2011	3307	KLW-3307	Aung	45	1	Kalaw	0	8888
5	MDY	08/03/2011	1859	MDY-1859	San	34	2	Mandalay	0	8888
6	NPT	24/10/2017	8648	NPT-8648	San	49	1	Nay Pyi Taw	6	1664
7	YGN	26/10/2015	7478	YGN-7478	Linn	42	2	MISSING	3	4278
8	KLW	10/04/2017	2480	KLW-2480	Aung	33	2	Kalaw	0	8888
9	MDY	14/01/2014	8740	MDY-8740	Myo	60	1	Mandalay	1	9999
10	RHK	09/01/2012	4064	RHK-4064	Aung	49	2	Rahkine	2	145
11	YGN	01/01/1900	4618	YGN-4618	San	52	2	Yangon	3	3543
12	MDY	04/10/2012	4200	MDY-4200	Linn	65	1	Mandalay	9	9999
13	RHK	20/11/2016	5630	RHK-5630	Minn	31	2	Rahkine	1	3845
14	TGG	08/03/2014	4812	TGG-4812	San	60	2	Taunggyi	4	5431
15	RHK	04/03/2018	0808	RHK-0808	Maung	34	2	Rahkine	0	8888

Figure 6.1.1 **All Data** display after data entry (`Ctrl + D` for window user)

Notes

In `pid` 4200, reason is blank while `regNum` is not. In practice, this case is not uncommon. What one should do in this situation is to cross-check this record with any other registry such as TB register.

In the case of `pid` 808, some value is there in `regNum` although reason is only for the diagnosis. This kind of mistake can also occur in real world. This should be corrected at the time of data entry.

Hence, data entry should also be trained about the data and its importance as well as common errors during entry and instructions to follow in such case.

Validation

After double entry, we will check whether anyone of the two data entry staffs make any mistakes or not. For the purpose of demonstration, let's introduce some errors to **B**'s project file, **form1_double.epx**.

- For `pid` 3307, change `ptSex` to **Female**.
- For `pid` 2480, change `reason` to **Follow-up at month 4**.
- For `pid` 4200, change `ptName` to **Minn**.

To validate the two files, open Manager. Go to **Documents** and choose **Compare Duplicate Files**. Click on **Add Files** and select the two files: `form1.epx` and `form1_double.epx`. (To select both files, press `Shift` and click on the files. OR you can add one by one.) You should see similar to the Figure 6.1.2.

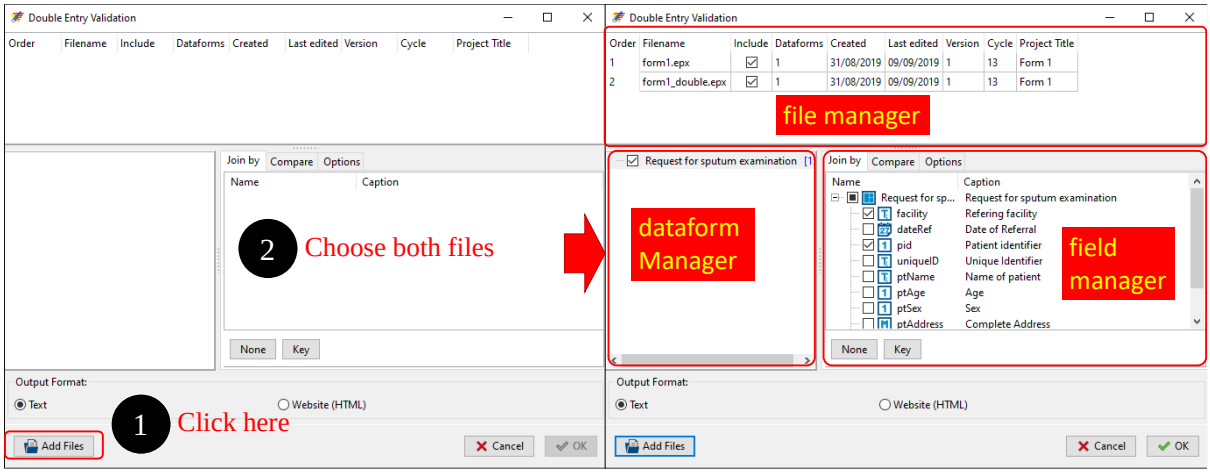


Figure 6.1.2 **Double Entry Validation Window** in EpiData Manager

The top longitudinal space is the place where input files for validation can be managed, thus let's call it **file manager**. The left space below it is to manage dataform (**dataform manager**) and the right one is for managing fields (**field manager**).

There are a few options to explore around in **field manager**. Default display is on **Join by** tab which basically tells EpiData which fields to take as key fields in order to match the two files. In our case, we already define two keys (`facility` and `pid`). As you may notice, EpiData automatically detects them.

Next two tabs are **Compare** and **Options**. In **Compare** tab, you can select fields of desire to compare between the two files. In **Options** tab, you can

- Exclude deleted records,
- Ignore case in text variables,
- Ignore missing records in duplicate file
- Add result variable

The last choice (add result variable) is a handy tool for data validation. This create a new variable of integer type with seven categories specific for validation. These categories are shown in Figure 6.1.3.

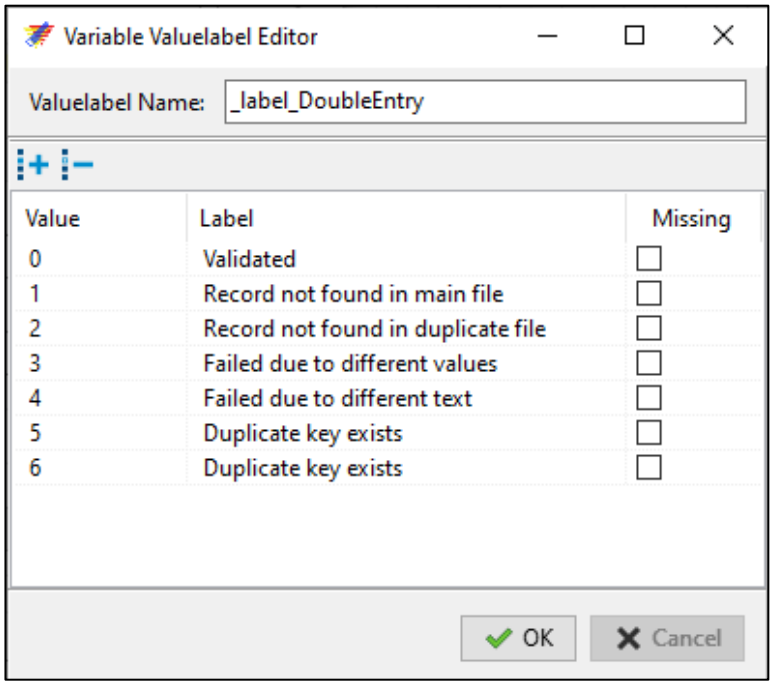


Figure 6.1.3 **Result Variable** specific for data validation

Before we generate the report, the last thing is to choose whether you want the report in a text file or more formatted and stylish **HTML** file. Both options are fine, but I recommend using **HTML** because of its stylish formatting and relatively better readability.

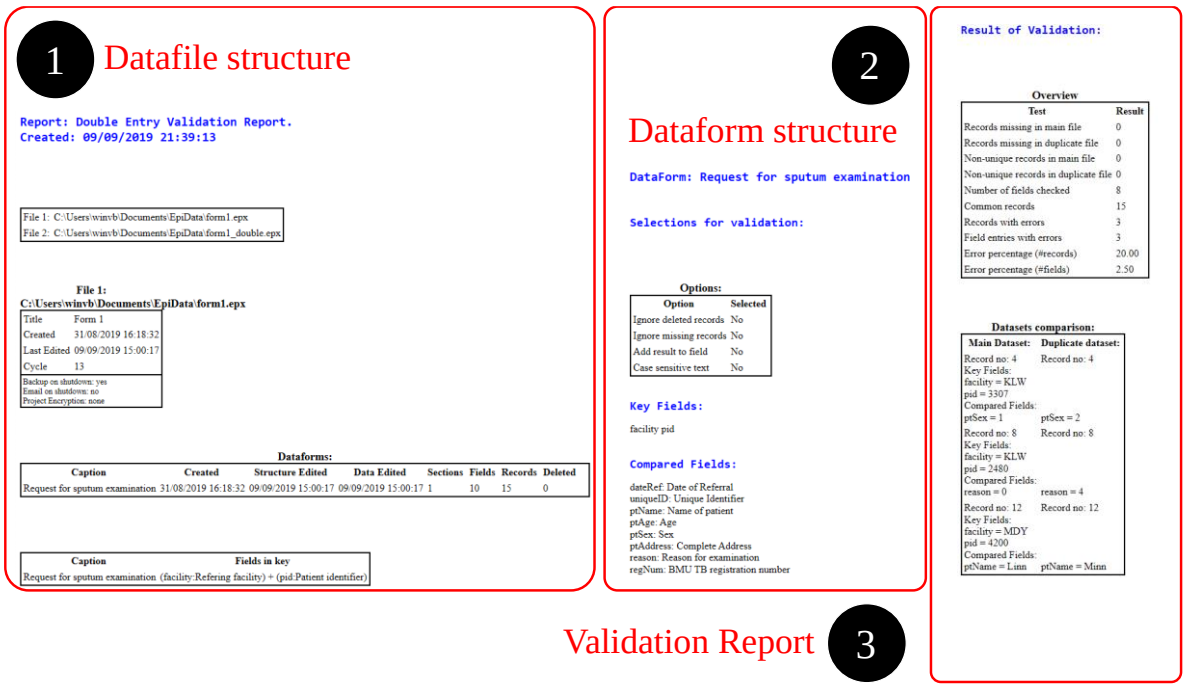


Figure 6.1.4 Report of data validation: information on (1) datafile structure, (2) dataform structure and (3) validation report

The final report is shown in Figure 6.1.4. It is intuitive as well as self-explanatory. It basically has three parts: ***datafile structure***, ***dataform structure*** and ***actual report of validation result***. The first two parts provide comprehensive information of the project file. However, for the purpose of data validation, the last part is the most important.

The **overview** box provides a very useful summary of figures with comparison between the two files. Based on this default option we chose earlier, EpiData compares missing records, non-unique records, number of fields checked, common records, records with errors and field entries with errors and percentages of records and field entries with errors.

The last box, **dataset comparison**, enumerates details of records with errors as shown in Figure 6.1.5. It is the actual place you have to look at in order to correct or update your data.

Datasets comparison:

Main Dataset:	**Duplicate dataset:**
Record no: 4 Key Fields: facility = KLW pid = 3307 Compared Fields: ptSex = 1	Record no: 4 ptSex = 2
Record no: 8 Key Fields: facility = KLW pid = 2480 Compared Fields: reason = 0	Record no: 8 reason = 4
Record no: 12 Key Fields: facility = MDY pid = 4200 Compared Fields: ptName = Linn	Record no: 12 ptName = Minn

Figure 6.1.5 Datasets comparison, the place to look at for correcting the dataset

So, we've got our report. What's next? We have to thoroughly cross-check with our paper-based records or other registries. Mark the records of the datasets with actual error.

Finalization

What we usually do at this stage is pick one file and modify whatever it is in that file. It's not a good practice. What we should do is to copy and paste one file and make change in the copied version. Picking one file is straightforward but remember that if you pick the file with less errors, your effort into correction will also be less. The newly copied version should now be named as `xxx_final.epx`. In our case, we will name it as `form1_final.epx`.

To enumerate the steps,

1. Save the report.
2. Print it out and put it beside your computer.
3. Copy and paste one of the two files.
4. Rename the copied file to `xxx_final.epx`.
5. Make necessary changes.
6. Save it.

That's it and we have finished **data double entry and validation**.

6.2 Exporting data

Data can be exported into five different formats, namely (1) comma-separated values (CSV), (2) stata, (3) SPSS, (4) Data Documentation Initiative (DDI) and (5) EpiData's default format (EPX). A CSV file is one of delimited text files that use a comma to separate data. It stores data in a tabular structure (rows and columns) in plain text. Each line is a record and fields are separated by commas.

Although this is the most commonly used file format to store data, it is not fully standardized. The idea of separating commas between fields can get complicated when the value in the field contains commas. Read more on **delimiter collision**. Another drawback is that even though the file extension is in `.csv`, this type of extension is used by other delimited text files, known as non-comma field separators, such as tab-delimited file or space-separated file. Some European countries use semi-colon as separator. This loose practice can cause problems in data exchange. If interested, read more on **RF4180 standard for CSV exchange**, **OKI fictionless tabular data package** and **internet W3C tabular data standard**. However, these topics are beyond the scope of this book and will not be discussed.

To export data, go to **Tools** and choose **Export**. Or Click on **Export…**

in the **progress toolbar**. Then choose your desired `.epx` file to export. In our case, we choose `form1.epx` as shown in Figure 6.2.1.

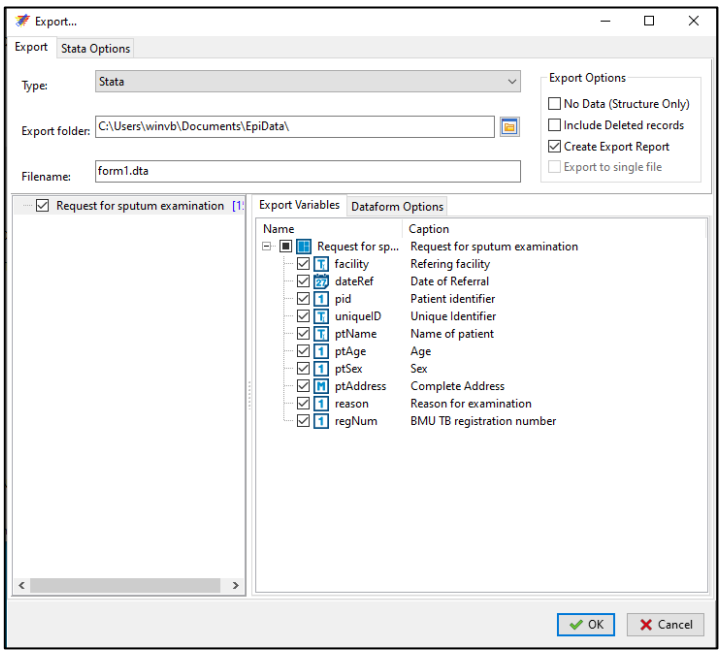


Figure 6.2.1 Export setting in EpiData Manager

There are two tabs: (1) Export (this is main interface and does not vary based on data type), and (2) Options (this provides additional settings based on data type). Generally, this is a very clean and intuitive interface. On left upper side, we can change (1) data type, (2) export folder and (3) exported filename.

On right upper side, there are four options:

(1) No Data (Structure only) – this is useful when you copy emptied project. This function can be an alternative for preparation of double data entry.

(2) Include Deleted Records – when a record is marked for deletion, you can exclude or include in the export even though the record is not physically deleted.

(3) Create export report

(4) Export to single file – this is handy when exporting relational dataform. This will be discussed in next chapter.

In the lower part of the window, you can choose dataforms on the left side. On the opposite, you can select variables of desire on **Export Variables** and specify the range of records to export in **Dataform Options**.

Stata

By default, EpiData points to **Stata 8,9** data type for data export, meaning that the exported file is compatible with old version of Stata software. EpiData now supports Stata data version up to 14.0.

Second, you can convert names of variables to either one of the three options: UPPERCASE, lowercase or Leave as it is. This becomes very handy for data analysis process.

Finally, you can choose to either export valuelabel or not. Value labels are a feature of data analysis using Stata.

CSV File

There is not much to change here, except some options to convert separator symbols which is not recommended to do at all. You can remove the heading or variables' name which is usually first row, but again it is not recommended to change anything in this type.

SPSS

Statistical package for social science (SPSS) is a commonly used software for data management and statistical analysis. Less options here, only one options to export Value Labels or not.

DDI

This will export data and meta-data in eXtensive Markup Language (XML). It is a text-based file and can store complicated data structure such as relational data. However, there are a whole session of debates out there on whether XML is the best option for storage and retrieval of data. Basically, it uses tags to identify the data which has been stored in an organized way. EpiData also uses its own grammars of different tags to structure and store data, which is a more advanced topic and will be discussed in the later chapter.

Even though it presents with several options to poke around, it is best to use the default option if you ever need the data in XML format.

EPX

Finally, you can just export data in EpiData project file. Since less is more, it is sometimes more efficient with less options.

Task 6.2. Export `form1.epx` using `EPX` file type into two files: (1) `form1_A.epx` which will contain record 1 to 8, and (2) `form1_B.epx` from record 9 to 15.

Note: We will use these two files for exercise in the next chapter **Appending Records**.

Technical Notes

EpiData XML File Format Specification (EPX) is a simplified EpiData specific adapted data file XML structure in 2009. It was based on The Data Documentation Initiative format (DDI) and the ODF standard. The purpose is to have a uniform way of saving and documenting data since there are a substantial number of varieties of alphabets, numbers and character sets on different types of platforms (Linux, Mac, Windows). (EpiData Wiki)

The essential requirements into developing the format narrowed down to the following facts:

- Speed of data retrieval and writing
- Cross-platform compatibility
- Support of Unicode and other character sets across different countries
- Minimal drawbacks from general data format specification requirements
- Support for export and import functionality.

The details on how the XML schema works are beyond the scope of this book. Read the usage of XML Schemas (also known as `.xsd` files) on the W3C school and the specification for XML schema files on the W3C. The full documentation for EpiData's schema file can be found here, which is an autogenerated list of html pages using the program <oXygen/> editor.

6.3 Appending records

It is pretty straightforward to append records. For an instance, you take a pile of 8 books and stack another 7 books onto that pile. As shown in Figure 6.3.1, Record A (Green color) is our base file where Record B (Yellow color) is added on to it.

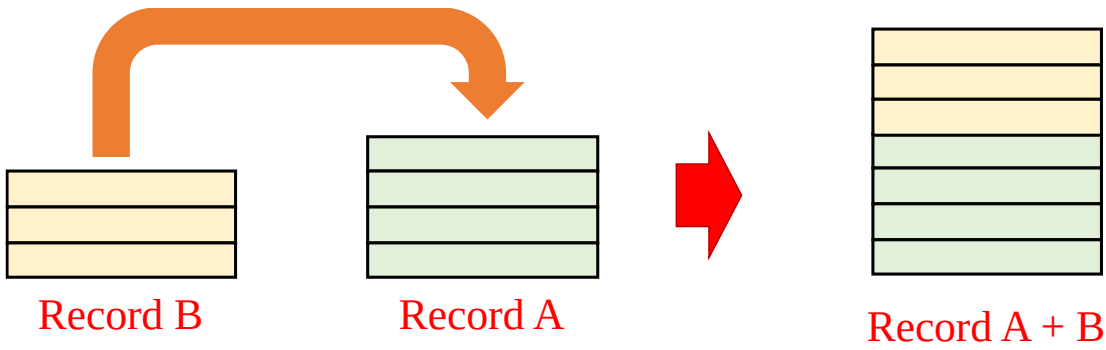


Figure 6.3.1 A visual representation of appending records

To append, go to **Tools** and choose **Append**. Then choose the base file. In our case, we choose `form1_A.epx` which we have prepared in previous chapter.

Now you will see the window as shown in Figure 6.3.2.

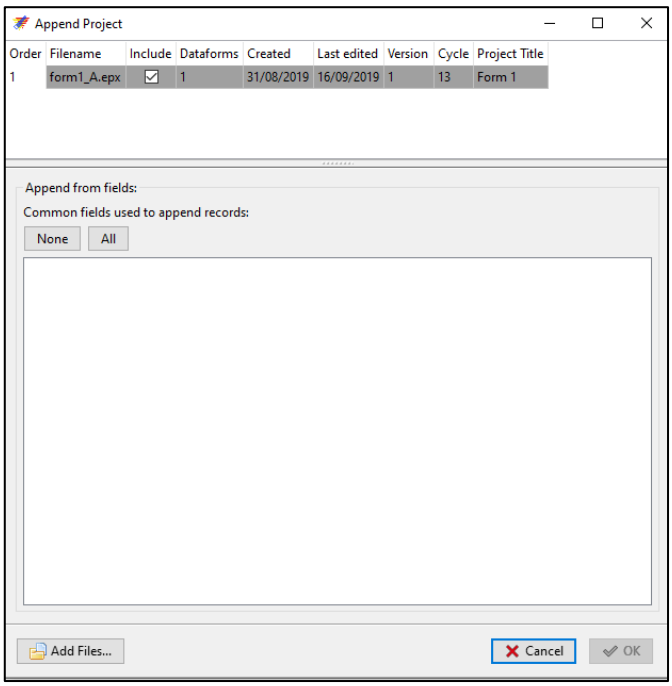


Figure 6.3.2 Choosing base file to append records in EpiData Manager

Next, click **Add Files** on the window to add more files. Choose `form1_B.epx` for our example. Then make sure to include both files by checking **include** box as shown in Figure 6.3.3. Select fields you desire in the lower part of the window and click **OK**.

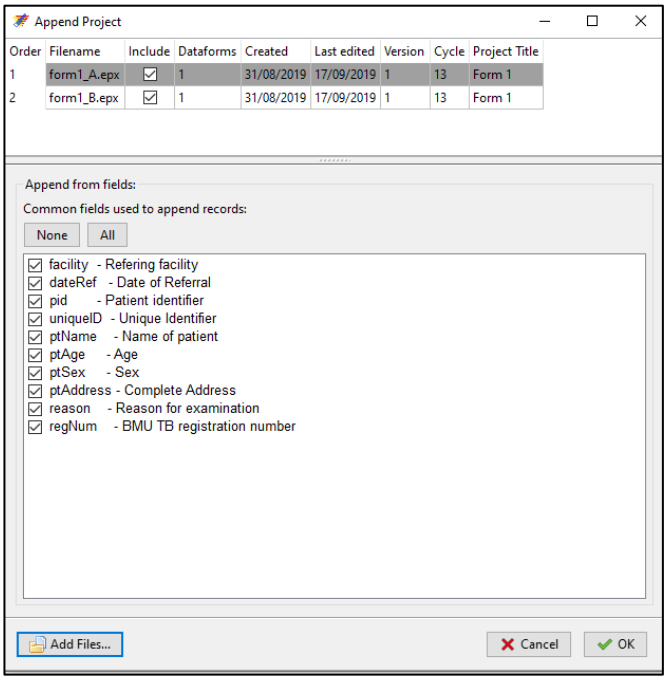


Figure 6.3.3 Appending records in EpiData Manager

You will now see the message from EpiData that our appending process is a success.

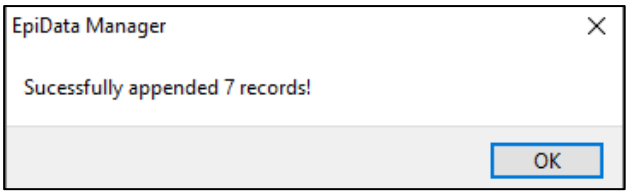


Figure 6.3.4 Success in appending records in EpiData Manager

But what happens if we append `form1_A.epx` to `form1.epx`? Just take a moment and think about it.

There are definitely 8 duplicates since `form1_A.epx` is a replica of `form1.epx` with records from 1 to 8. Since we already defined key fields or unique index for the file, EpiData immediately lets us know that there are exactly 8 duplicates and asks if we wish to continue appending remaining dataforms. Figure 6.3.5 illustrates this.

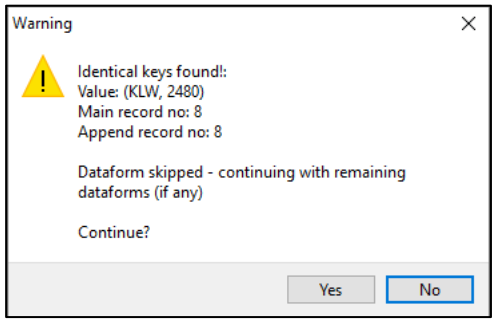


Figure 6.3.5 Warning when appending records with duplicates

Task 6.3. Append `form1_A.epx` to `form1.epx` and observe the warning message.

6.4 File backup

Backup represents the process of creating and storing copies of data to protect against data loss. Typically, it involves storing proper backup copies in a separate location, medium or system. Copies can then be restored in case of primary data failures which may occur due to hardware or software failure, data corruption, or a human-caused event, such as a malicious attack (virus or malware), or accidental deletion of data.

As a good clinical practice, it is recommended to make backup copies on a consistent, regular basis to minimize the amount data lost between backups. The more time elapsed between backup copies, the higher chance to lose data when recovering from a backup. Therefore, retaining multiple copies of periodic data guarantees the insurance that cannot affected by data corruption or malicious attacks.

Storage drives

Not for long, we have invested a lot in independent storage drives. We have entered into an era where drives capable of terabyte storage can be purchased just around the corner. These devices are invaluable to protect your data.

We have one more option, offsite server. These days, our data is in the cloud, but don't look up to the sky! Basically, these storages are provided in volumes by organization or core IT environment and costs for services and maintenance are coming down fast. So, most people can afford these services. For an instance, Google is one of the biggest IT company in the world that provide affordable online backup solutions. Dropbox, Mega, Microsoft's OneDrive and pCloud are some examples. Considering these options will definitely benefit you on the long run.

Zipped EPX File

In EpiData, there is no explicit backup function or button. At this point, you may perhaps notice that there is a folder called **backup** in the same directory. This is automatically created by EpiData in `.epz` format when you open the project in EntryClient and enter data. This is a zipped version of `.epx` format with encryption in **Advanced Encryption Standard** (AES). AES is commonly used worldwide, and its encryption capability supersedes the Data Encryption Standard (DES) published in 1977. AES uses a symmetric-key algorithm, meaning the same key is used for both encrypting and decrypting the data. United States Government announced in 2003 that AES could be used to protect classified information. There has been several known attacks to the security offered by AES. This leads to the conclusion that EpiData provides very good data security against hacking, if not the best.

A simple way of demonstrating this security process is using **Notepad** in Window or **textEdit** on Mac OS. You can use any text editors to do this.

Now open `form1.epx` with **Notepad** or **textEdit**. The data content inside the file may look messy but you can clearly see it, as shown in Figure 6.4.

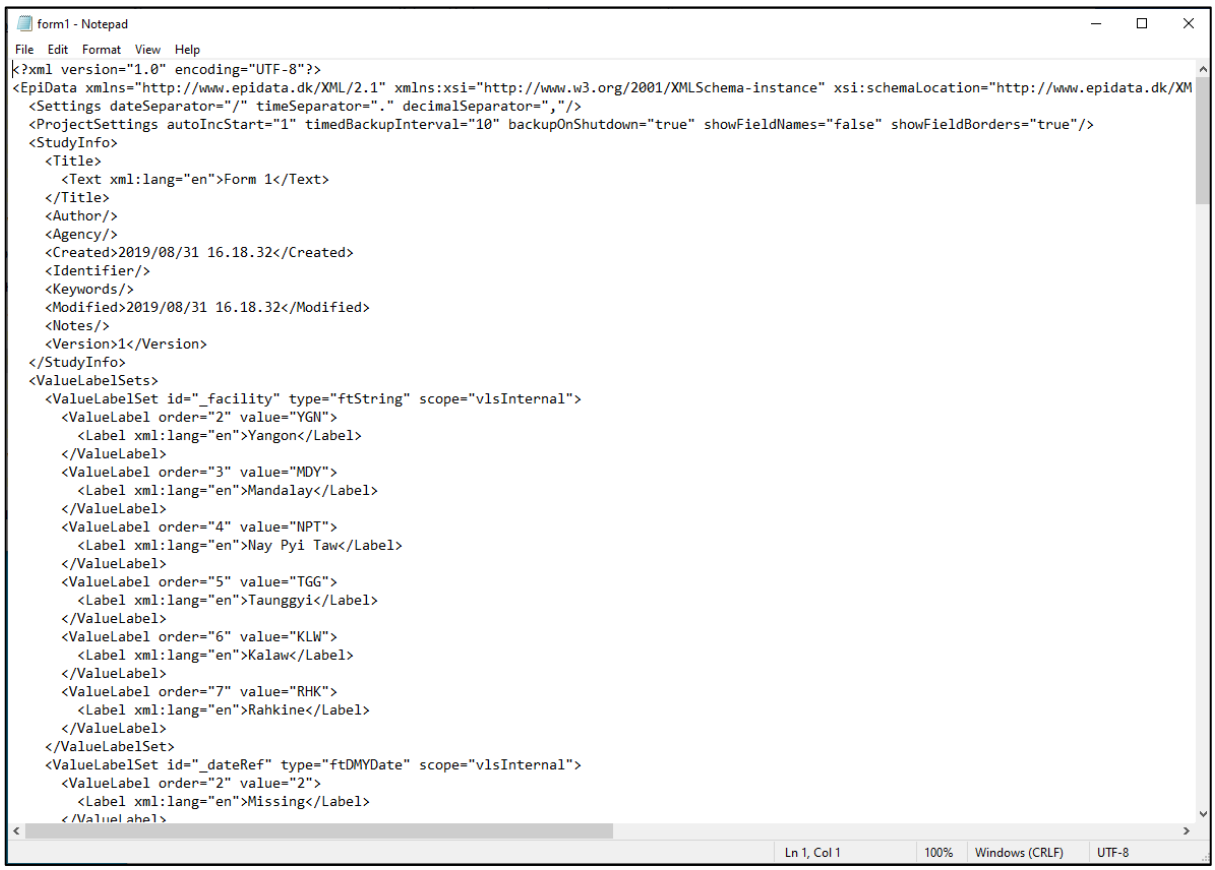


Figure 6.4 EPX file in Notepad (Window)

6.5 Archiving files with encryption

We now know that encryption in EpiData is pretty strong. Can we make use of this further? The answer is yes! EpiData kindly puts functionality of creating and extracting such zipped archives with or without password. Let's try this.

Go to **Tools** and choose **Create Archive**. As you can see in Figure 6.5.1, you can either choose the whole folder (also include sub-folders), use filters to specify file types you desire or select a single file. In our case, let's choose `form1.epx` as single file.

One important thing to note here is that if you do not encrypt with passwords, EpiData will just zip the file or files, which is, of course, not encrypted and therefore, not secure for data sharing. So, let's encrypt this with our usual simple password, 1234. **DO NOT USE THIS IN REAL PROJECT**! IT'S THE WORST CHOICE OF PASSWORDS! Check this link for the list of common passwords and this for top 100 worst passwords featured in Security Magazine.

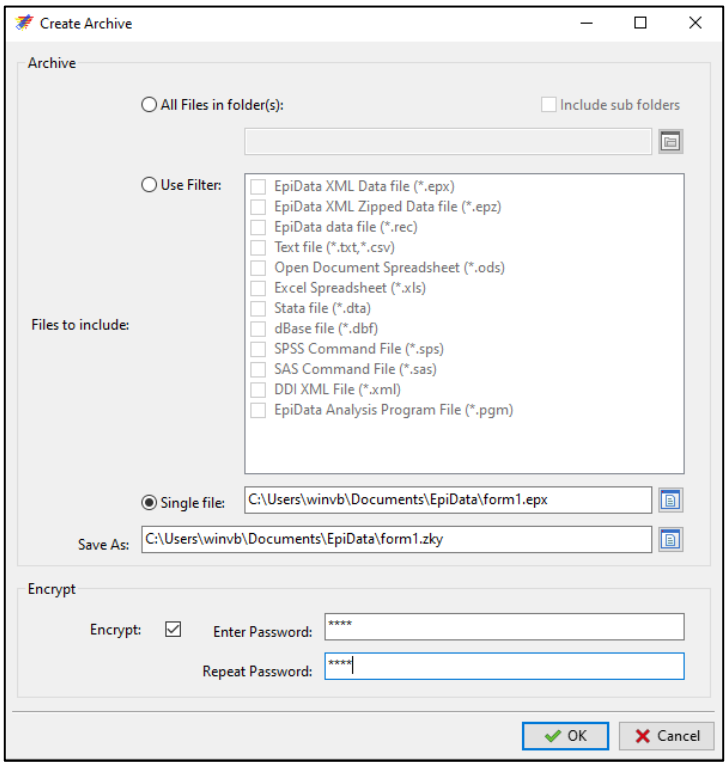


Figure 6.5.1 Archiving data with encryption

Make sure to notice that the file format is not the same here as before. It is in `.zky` format. But the principle is the same. In order to get the data inside the file, you need EpiData Manager or at least the password to decrypt it. The former backup file type can be opened in

EpiData Manager or EntryClient without the need for password. Let's try to open the file in Notepad. See Figure 6.5.2 for gibberish contents inside `.zky` and `.epz` files opened in Notepad.

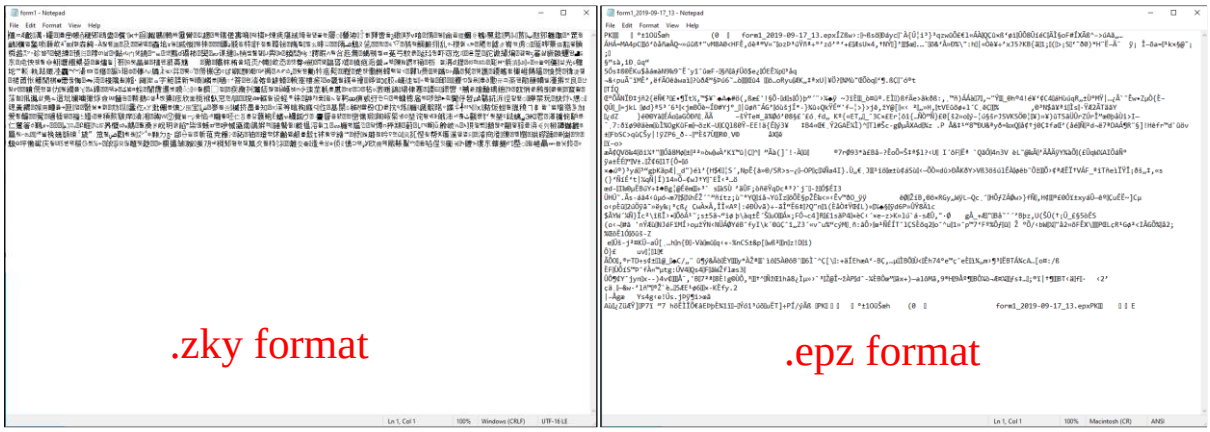


Figure 6.5.2 Opening `.zky` and `.epz` files in Notepad

Now let's try extracting the file to get our original data. Before doing this, let's rename our current `form1.epx` to `form1_ORIGINAL.epx`. Go to **Tools** and Choose **Extract Archive**. As shown in Figure 6.5.3, choose `form1.zky`, check both **Decrypt** and **Unzip**, and key in our notorious password, 1234. Click **OK**. You may select desired destination folder if you want.

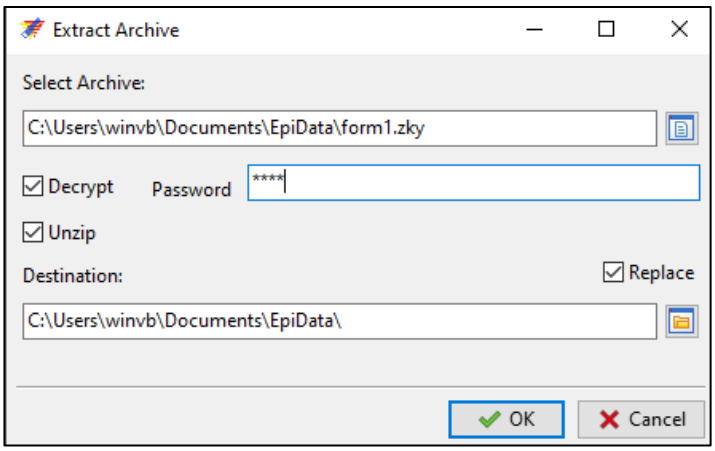


Figure 6.5.3 Extracting archived file

Now we get our data back from archived file. An alternative is to open archived files from EpiData directly. It has some disadvantages and is generally not recommended.

Chapter 7. Data documentation

7.1 Generating Report

This is an extended functionality of EpiData to generate codebook which was discussed in Chapter 3.

7.2 Comparing files for duplicates

7.3 Count records

7.4 Data content validation

Chapter 8. Creating relational database

8.1 Relational database
8.2 Creating relational dataform
8.3 Enter relational data
8.4 Deleting and Exporting relational data

Chapter 9. User access control system

9.1 Setting single password
9.2 User access control
9.3 Defining roles and rights
9.4 Access log Overview
9.5 Removing the control

Chapter 10. Advanced properties of dataforms

Chapter 11. Advanced settings

11.1 Version control

Chapter 12. EpiData and R

Annexure: Shortcut keys

Shortcut	Description of action
Dropdown Menu: File	
Alt + F	**Open the dropdown menu**
Ctrl + N	Create a new project
Ctrl + O	Open an existing project
Ctrl + Shift + 1	Open the first most recent project
Ctrl + Shift + 2	Open the second most recent project
Ctrl + Shift + 3	Open the third most recent project
….	And so on
Ctrl + S	Save current project
Ctrl + Shift + S	Save current project under a different name
Ctrl + F4	Close the project
Ctrl + I	Import external file
Ctrl + Shift + I	Import from clipboard
Ctrl + P	Print the dataform
Alt + F4	Close the software
Dropdown Menu: Edit	
Alt + E	**Open the dropdown menu**
Ctrl + Z	Undo the action done
Ctrl + Shift + Z	Redo the action done
Ctrl + X	Cut
Ctrl + C	Copy
Ctrl + V	Paste
Ctrl + Shift + Left	Align the field to the left
Ctrl + Shift + Right	Align the field to the right
Ctrl + Shift + Up	Align the field to the top
Ctrl + Shift + Down	Align the field to the bottom
Ctrl + Shift + A	Open the “Alignment” box
Alt + S	Open the “Preferences” box
Ctrl + Shift + 0	Put the “EpiData Manager” window in default position
Dropdown Menu: Project	
Alt + R	**Open the dropdown menu**
Alt + P	Open the “Project Properties” box
Alt + V	Open the “Value Labels” box
Dropdown Menu: User Access	
Alt + U	**Open the dropdown menu**

Ctrl + G	Define User Group
Ctrl + U	Define User
Ctrl + E	Define the entry rights for each user
Ctrl + L	View the log
Dropdown Menu: Dataform	
Alt + A	**Open the dropdown menu**
Ctrl + D	Browse the dataset of current dataform
Dropdown Menu: Document	
Alt + D	**Open the dropdown menu**
Dropdown Menu: Help	
Alt + H	**Open the dropdown menu**
Additionals	
Alt + Q	Close the software
Alt + C	Close popup windows
L	Open the menu of "Select Project" from the progress toolbar
Enter	When the dataform is active, press enter to open the "Dataform Properties" box.
Delete	Delete the selected field/heading/section with confirmation
Shift + Delete	Delete the selected field/heading/section without confirmation
Home	Select the top field/heading/section
End	Select the bottom field/heading/section
Page Up	Select the field/heading/section on the previous page
Page Down	Select the field/heading/section on the next page
Up Arrow	Select the previous field/heading/section
Down Arrow	Select the next field/heading/section
Ctrl + minus	Expand the columns in "Log", "Valuelabel Editor"
Shortcuts enabled when dataform is selected	
F2	Rename the dataform
1	Insert New **Integer** Field and open the "Variable Properties" box
2	Insert New **Float** Field and open the "Variable Properties" box
3	Insert New **String** Field and open the "Variable Properties" box
4	Insert New **Date** Field and open the "Variable Properties" box
5	Insert New **Label** Field and open the "Variable Properties" box
6	Insert New **Heading** Field and open the "Variable Properties" box

When pressed these numbers altogether with "Shift", insert a new respective field without opening the "Variable Properties" box.

Example: press Shift + 1 >>> insert an integer input box without opening the "Variable Properties" box".

Using Keyboards for those Menus that do not have shortcuts

Step 1: Press the shortcut for the dropdown menu you desire.

Step 2: Press the initial letter of the submenu you desire.

If there are several submenus that start with the same initial letter, perform step 1 and press the "initial letter" repeatedly to get the submenu you desire.

Step 2.0: Press "Enter".

Example 1: Suppose you want to open the "Project Properties".

Step 1: Press "Alt + R".

Step 2: Press "P".

Example 2: Suppose you want to open "Preferences".

Step 1: Press "Alt + E".

Step 2: Press "P" seven times.

Step 3: Press "Enter".